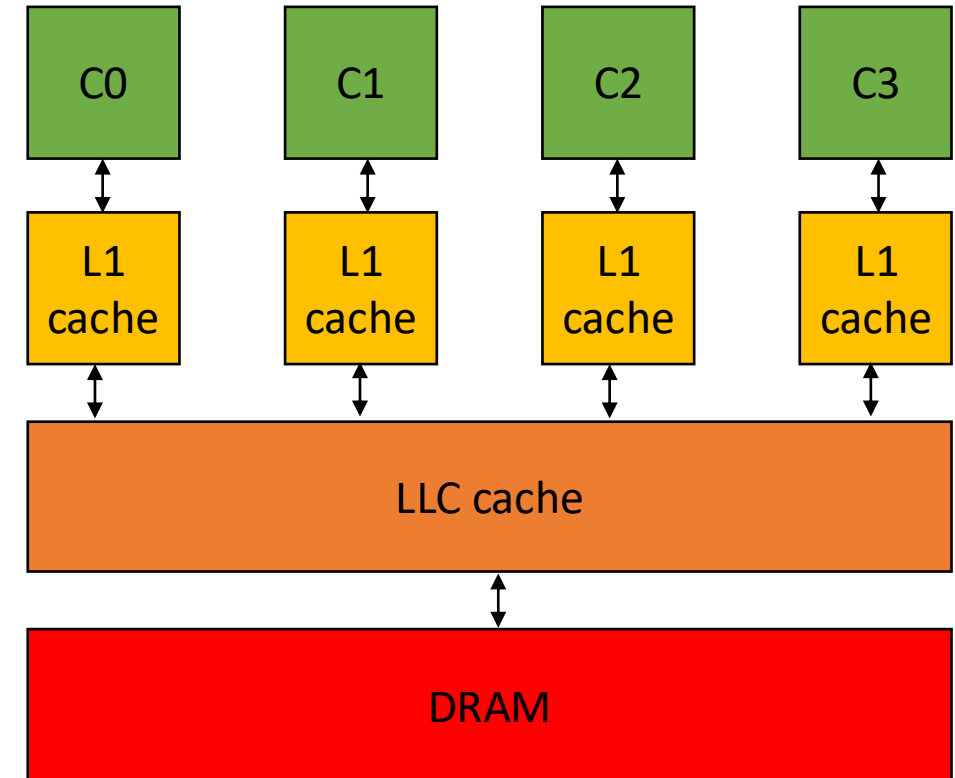


CSE113: Parallel Programming

- **Topic:** Architecture and Compiler Overview
 - Programming Language to ISA compilation
 - 3-address code
 - multiprocessors
 - memory hierarchy



Announcements

- Homework 1 released tonight!
 - A week to do it
 - 3 free late days
- Solutions require a design doc.
 - Not harshly graded but liable to lose points for low-effort
 - Forces you to think about your solution before you start

Announcements

- Instructor and TAs: Office hours announced on the webpage
- Tutors: Office hours one announced, another announced by Thursday.

Review

In a perfect world...

- Historically this worked well



The negotiators:
Specifications
Compiles
Runtimes
Interpreters

- Dennard's scaling:
 - Computer speed doubles every 1.5 years.

2003
700 MHz



3x *increase*
over 4 years

2007
2.1 GHz



However...

These trends slowed down in ~2007



The negotiators:
Specifications
Compiles
Runtimes
Interpreters



2007
2.1 GHz

1.2x increase
over 10 years

2017
2.5 GHz



2 cores



4 cores

Python?

- Great language for scripting
 - We will use it to automate experiments in this class
- The GIL (global interpreter lock) restricts parallelism significantly.
 - Makes the language safe
- TensorFlow and Pytorch?
 - Wrappers around low-level kernels that execute outside of the python interpreter

Lecture Schedule

- Overview - why do we need a lecture on compilation and architecture?
- **Compilation** - *How do we translate a program from a human-accessible language to a language that the processor understands?*
- Architecture - How do processors execute programs?
- Example

Compilation:

Language



ISA



Compilation:

Language



```
int add(int a, int b) {  
    return a + b;  
}
```

ISA



Compilation:

Language



```
int add(int a, int b) {  
    return a + b;  
}
```

*If we didn't have
computers, would this
mean anything?*

ISA



Compilation:

Language



```
int add(int a, int b) {  
    return a + b;  
}
```

Officially defined by the specification

ISO standard: costs \$200

~1400 pages

ISA



Compilation:

Language



```
int add(int a, int b) {  
    return a + b;  
}
```

Officially defined by the specification

ISO standard: costs \$200

~1400 pages

ISA



Official specification

Intel provides a specification: *free*

2200 pages

Compilation:

Language



```
int add(int a, int b) {  
    return a + b;  
}
```

Officially defined by the specification

ISO standard: costs \$200

~1400 pages

ISA



???

Official specification

Intel provides a specification: *free*

2200 pages

Compilation:

Language



```
int add(int a, int b) {  
    return a + b;  
}
```

Officially defined by the specification

ISO standard: costs \$200

~1400 pages



```
add(int, int): # @add(int, int)  
push rbp  
mov rbp, rsp  
mov dword ptr [rbp - 4], edi  
mov dword ptr [rbp - 8], esi  
mov eax, dword ptr [rbp - 4]  
add eax, dword ptr [rbp - 8]  
pop rbp  
ret
```

Official specification

Intel provides a specification: *free*

2200 pages

Compilation:

Language



```
int add(int a, int b) {  
    return a + b;  
}
```

Officially defined by the specification

ISO standard: costs \$200

~1400 pages



```
add(int, int):  
    sub sp, sp, #16  
    str w0, [sp, #12]  
    str w1, [sp, #8]  
    ldr w8, [sp, #12]  
    ldr w9, [sp, #8]  
    add w0, w8, w9  
    add sp, sp, #16  
    ret
```

How about a more complicated program?

Quadratic formula

How about a more complicated program?

Quadratic formula

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

How about a more complicated program?

Quadratic formula

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

```
x = (-b - sqrt(b*b - 4 * a * c)) / (2*a)
```

How about a more complicated program?

Quadratic formula

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

```
x = (-b - sqrt(b*b - 4 * a * c)) / (2*a)
```



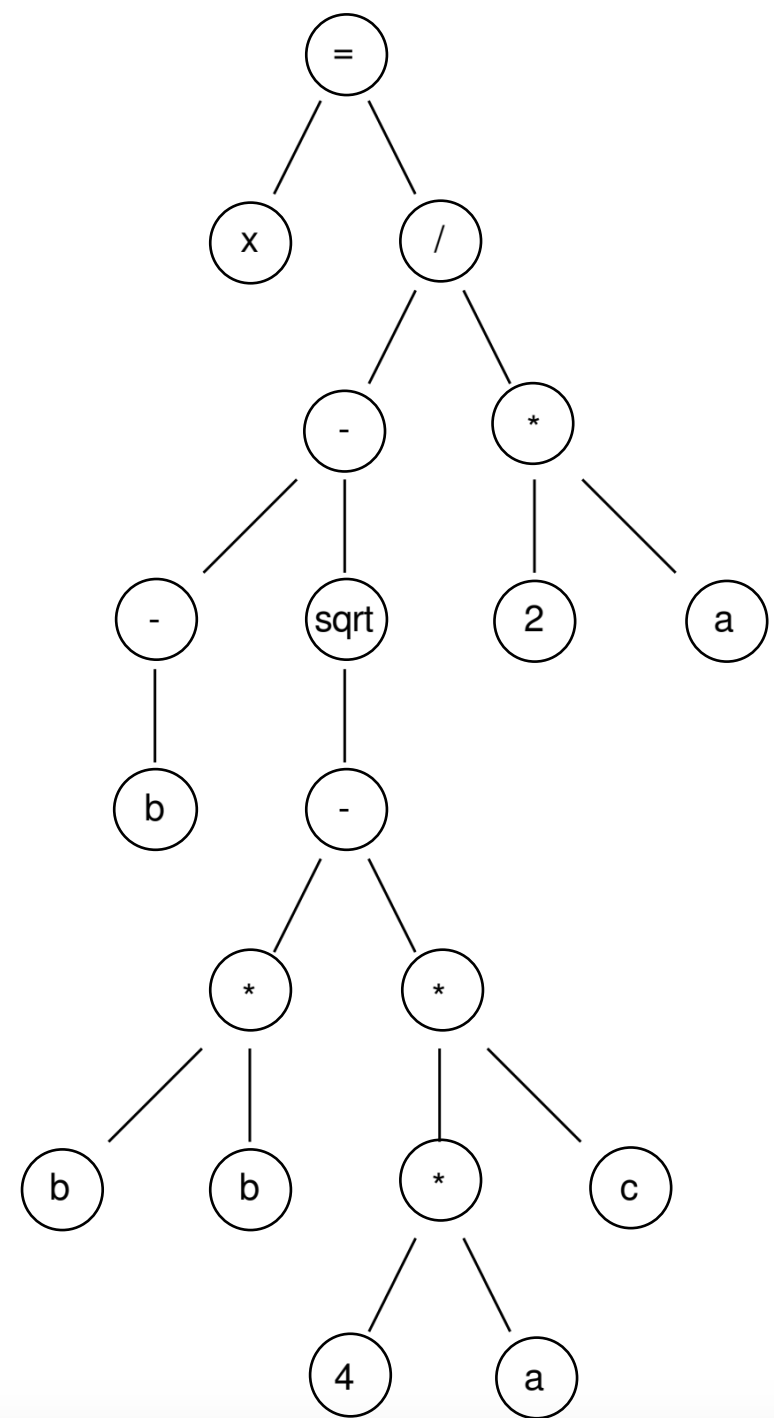
Official specification

Intel provides a specification: *free*
2200 pages

There is not an ISA instruction that combines all these instructions!

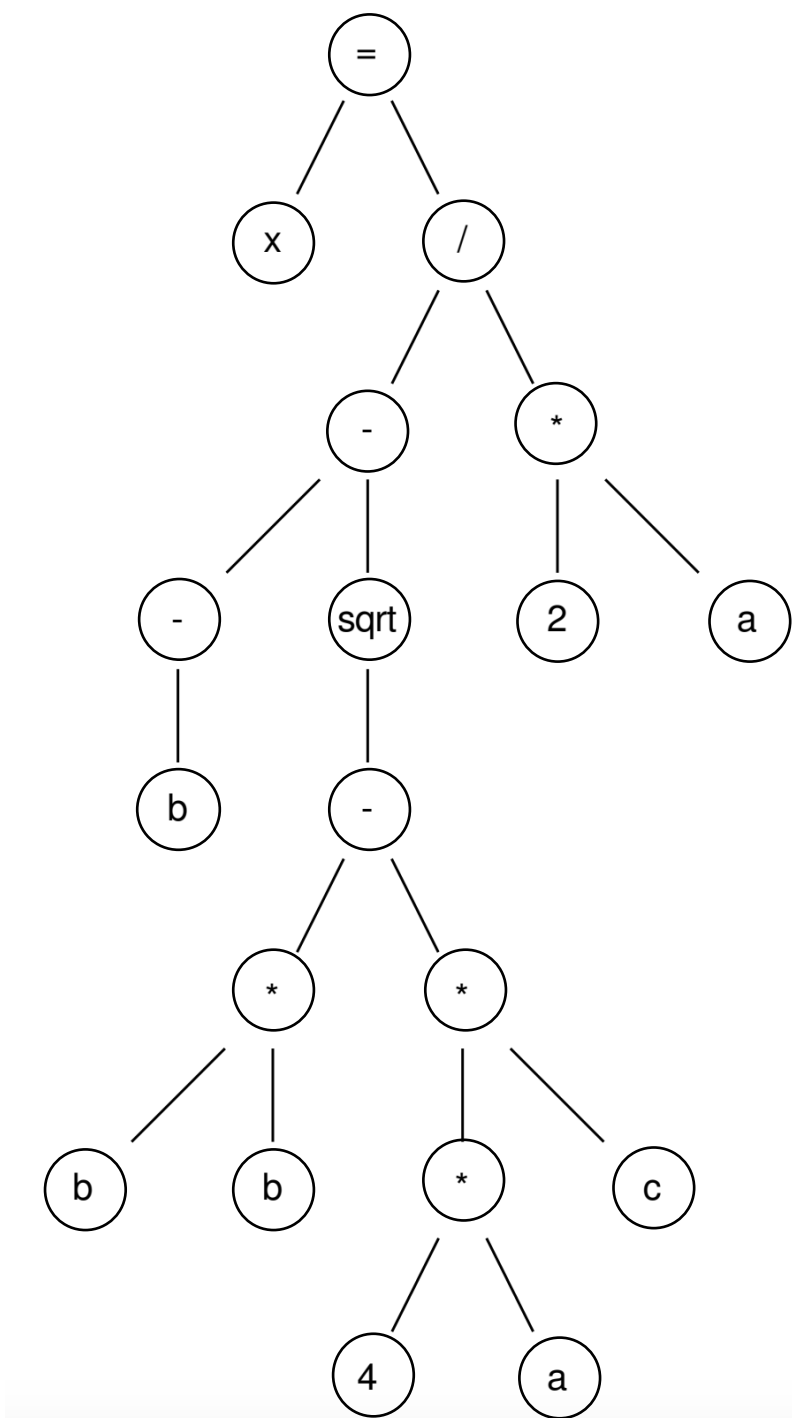
$x = (-b - \sqrt{b*b - 4 * a * c}) / (2*a)$

A compiler will turn this into an
abstract syntax tree (AST)



Simplify this code:

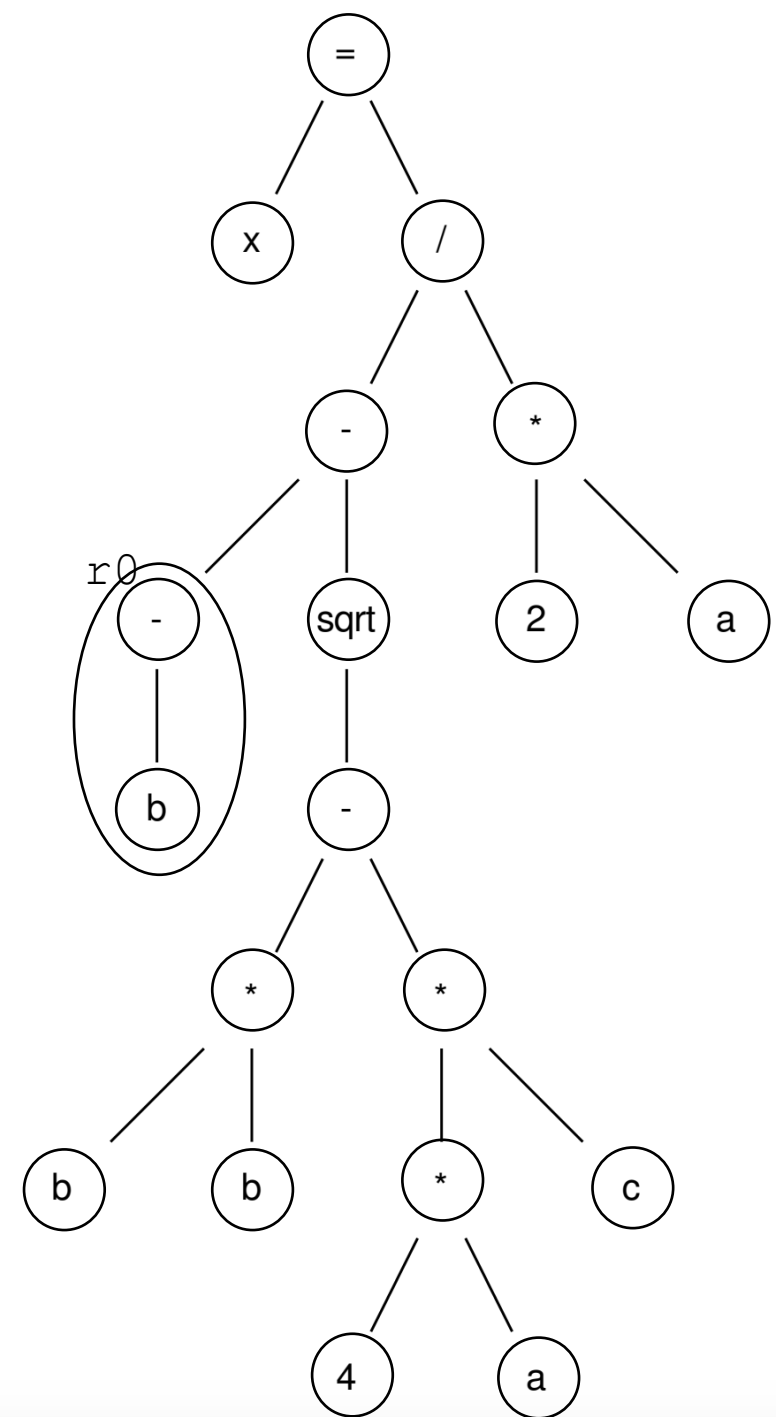
Post-order traversal, using temporary variables



Simplify this code:

post-order traversal, using temporary variables

```
r0 = neg(b) ;
```



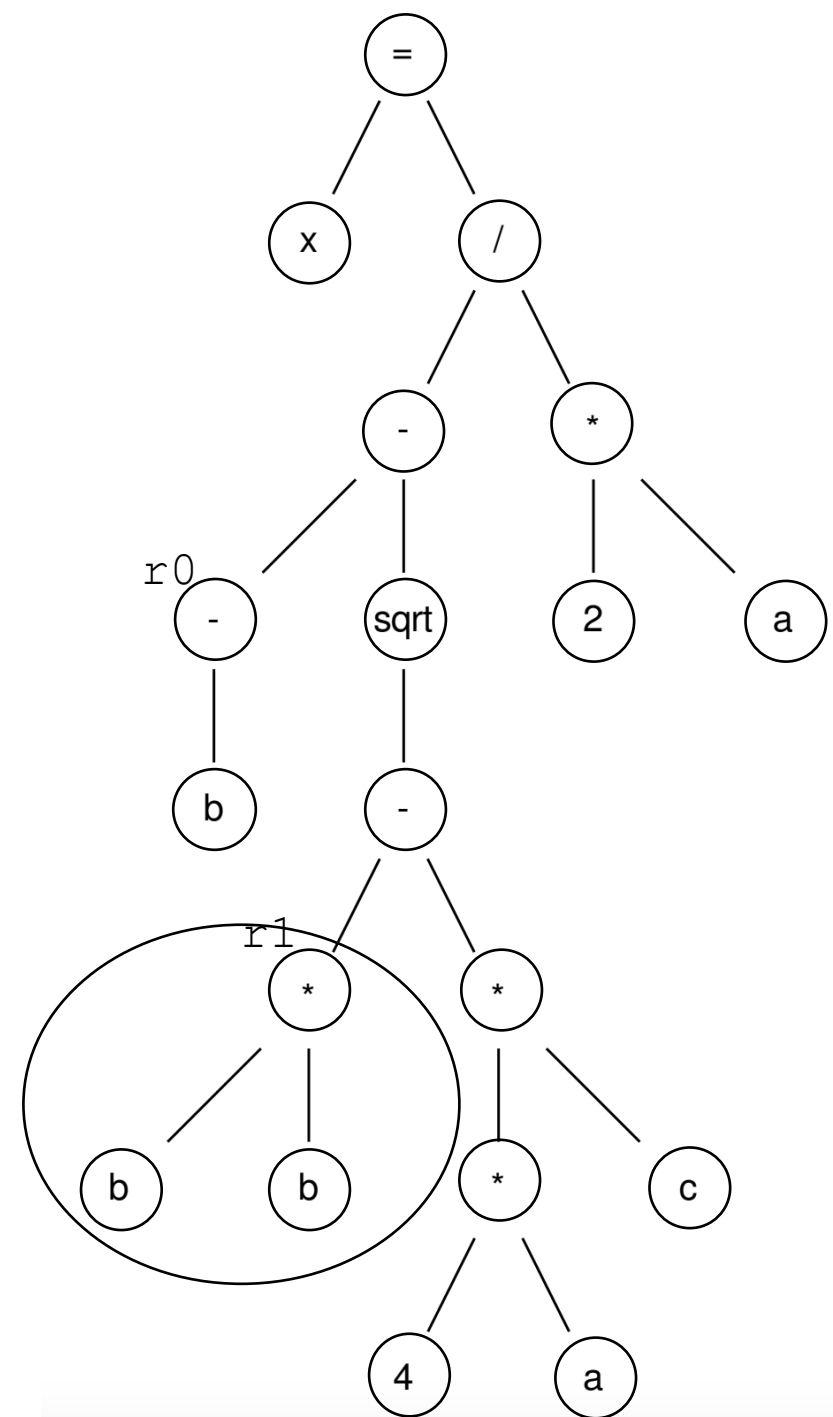
Simplify this code:

post-order traversal, using temporary variables

```

r0  =  neg (b) ;
r1  =  b  *  b ;

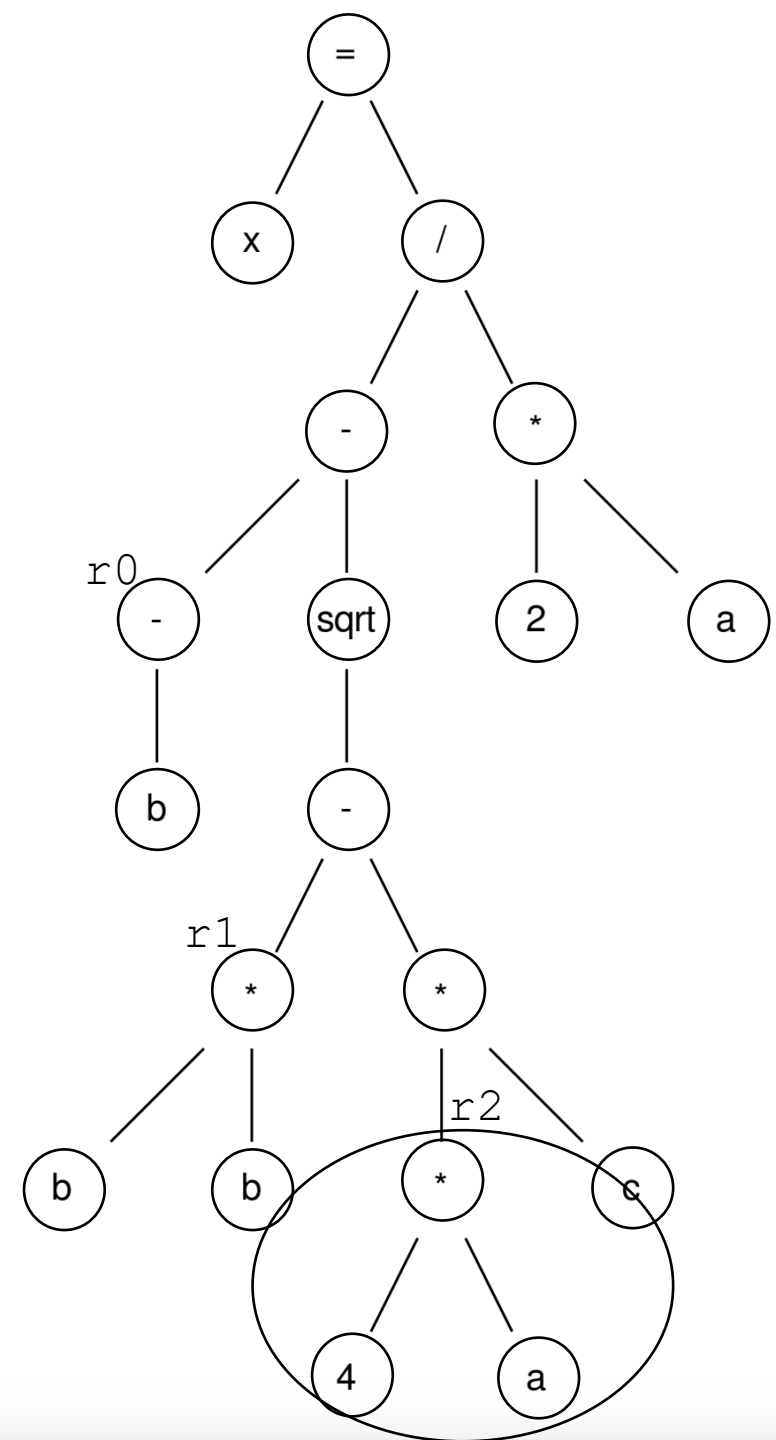
```



Simplify this code:

post-order traversal, using temporary variables

```
r0 = neg(b) ;  
r1 = b * b ;  
r2 = 4 * a ;
```



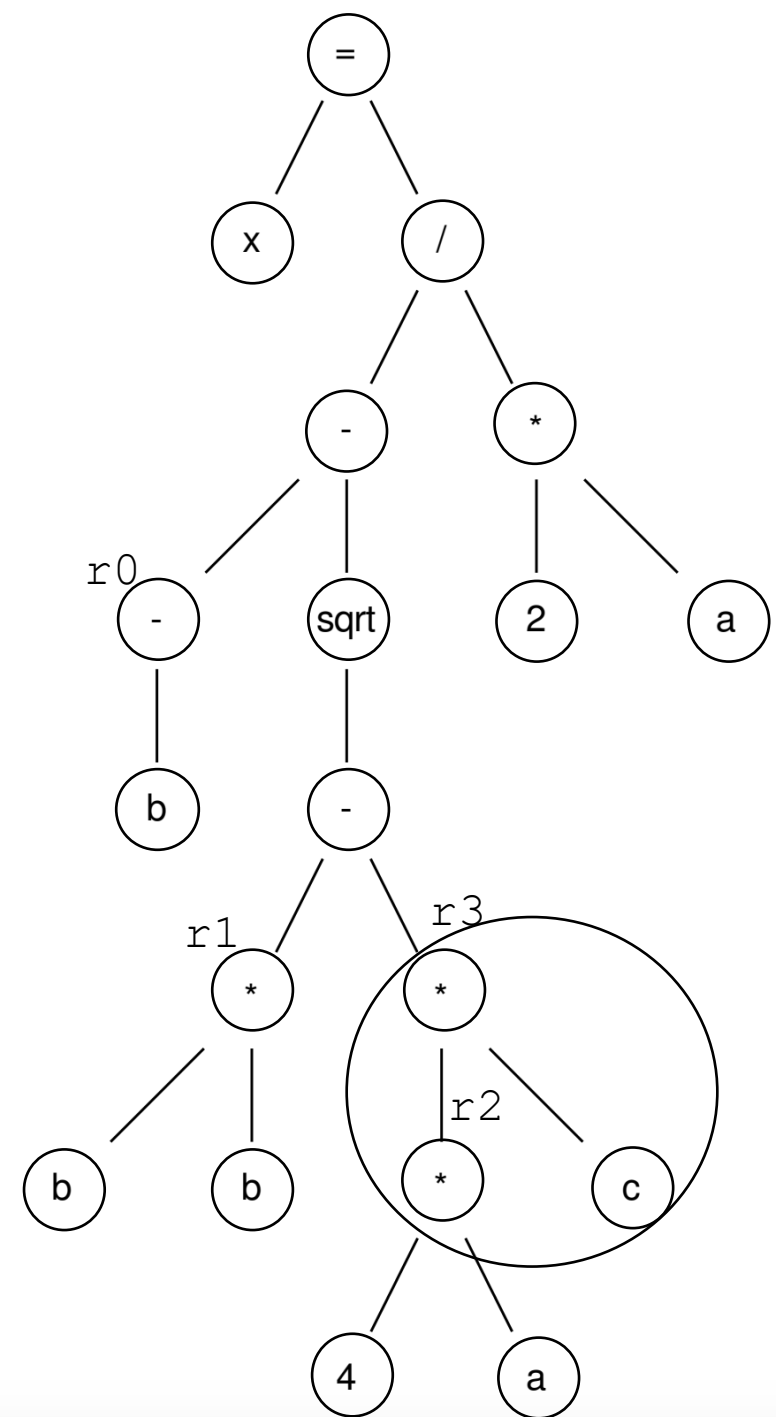
Simplify this code:

post-order traversal, using temporary variables

```

r0 = neg (b) ;
r1 = b * b;
r2 = 4 * a;
r3 = r2 * c;

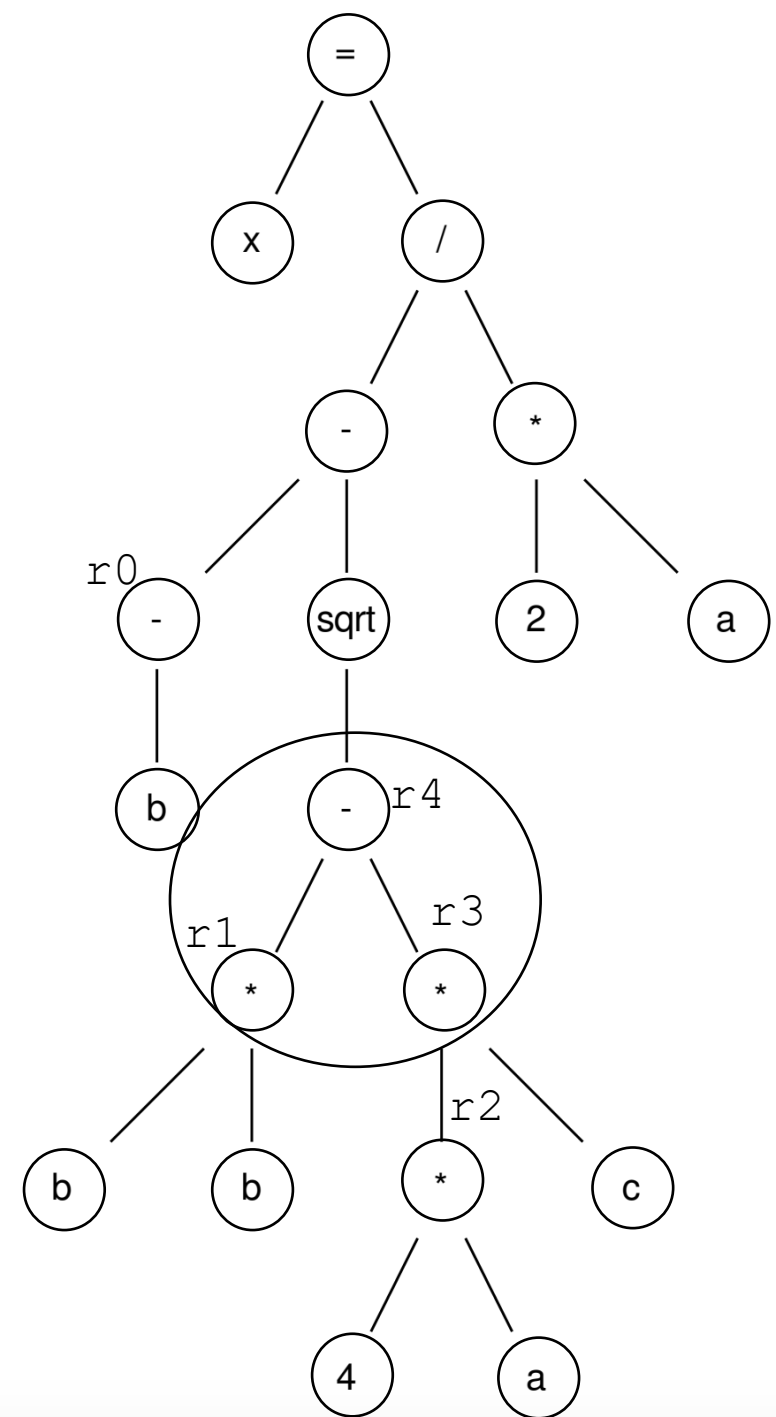
```



Simplify this code:

post-order traversal, using temporary variables

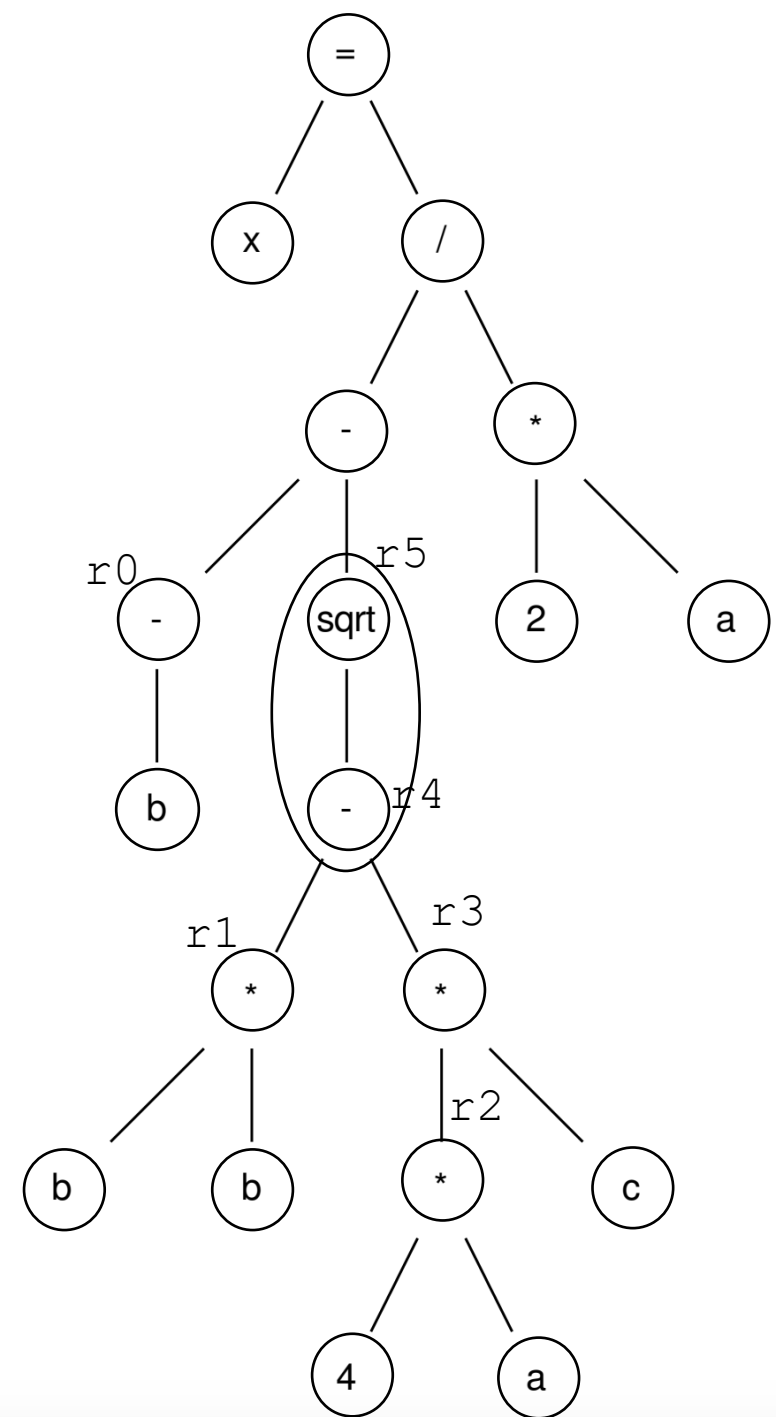
```
r0 = neg(b) ;  
r1 = b * b ;  
r2 = 4 * a ;  
r3 = r2 * c ;  
r4 = r1 - r3 ;
```



Simplify this code:

post-order traversal, using temporary variables

```
r0 = neg(b) ;  
r1 = b * b ;  
r2 = 4 * a ;  
r3 = r2 * c ;  
r4 = r1 - r3 ;  
r5 = sqrt(r4) ;
```

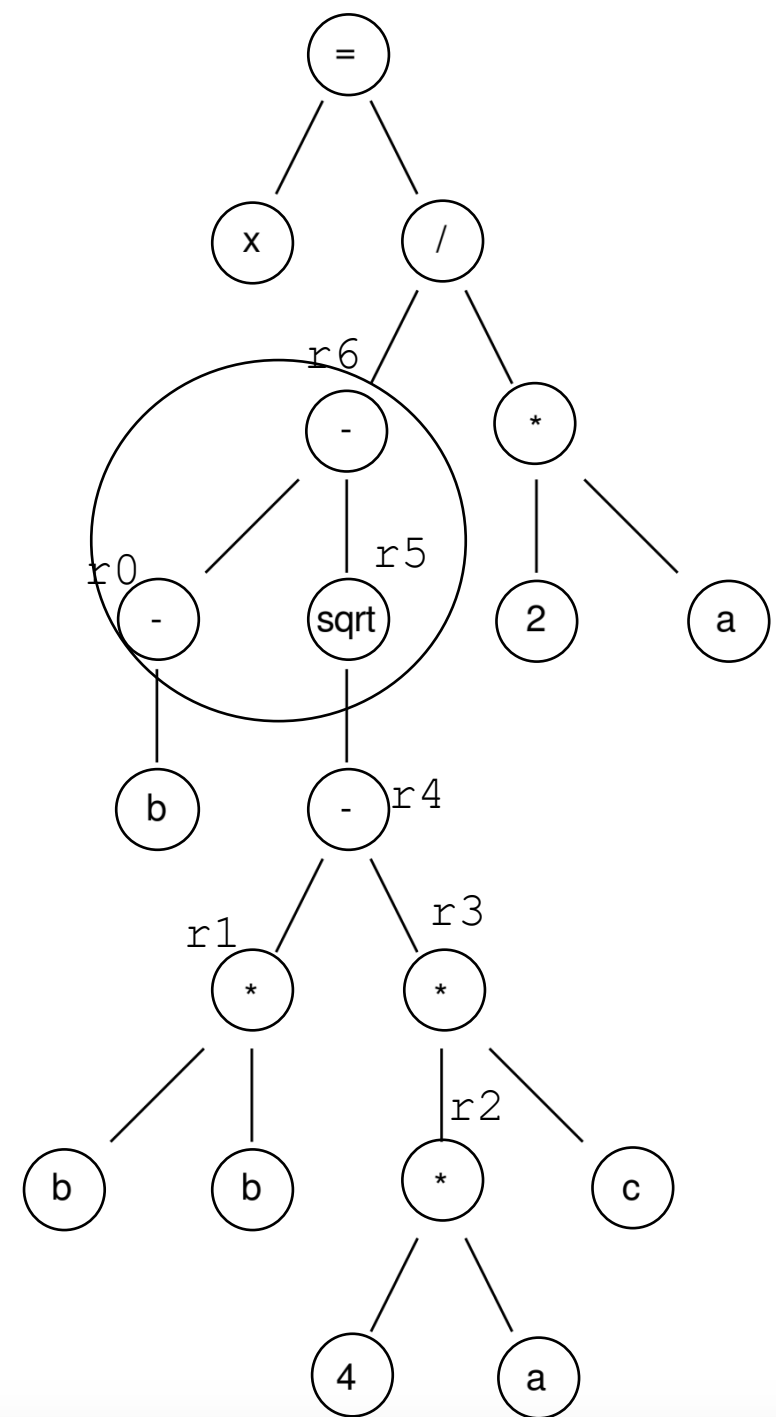


post-order traversal, using temporary variables

```

r0  =  neg (b) ;
r1  =  b  *  b ;
r2  =  4  *  a ;
r3  =  r2  *  c ;
r4  =  r1  -  r3 ;
r5  =  sqrt (r4) ;
r6  =  r0  -  r5 ;

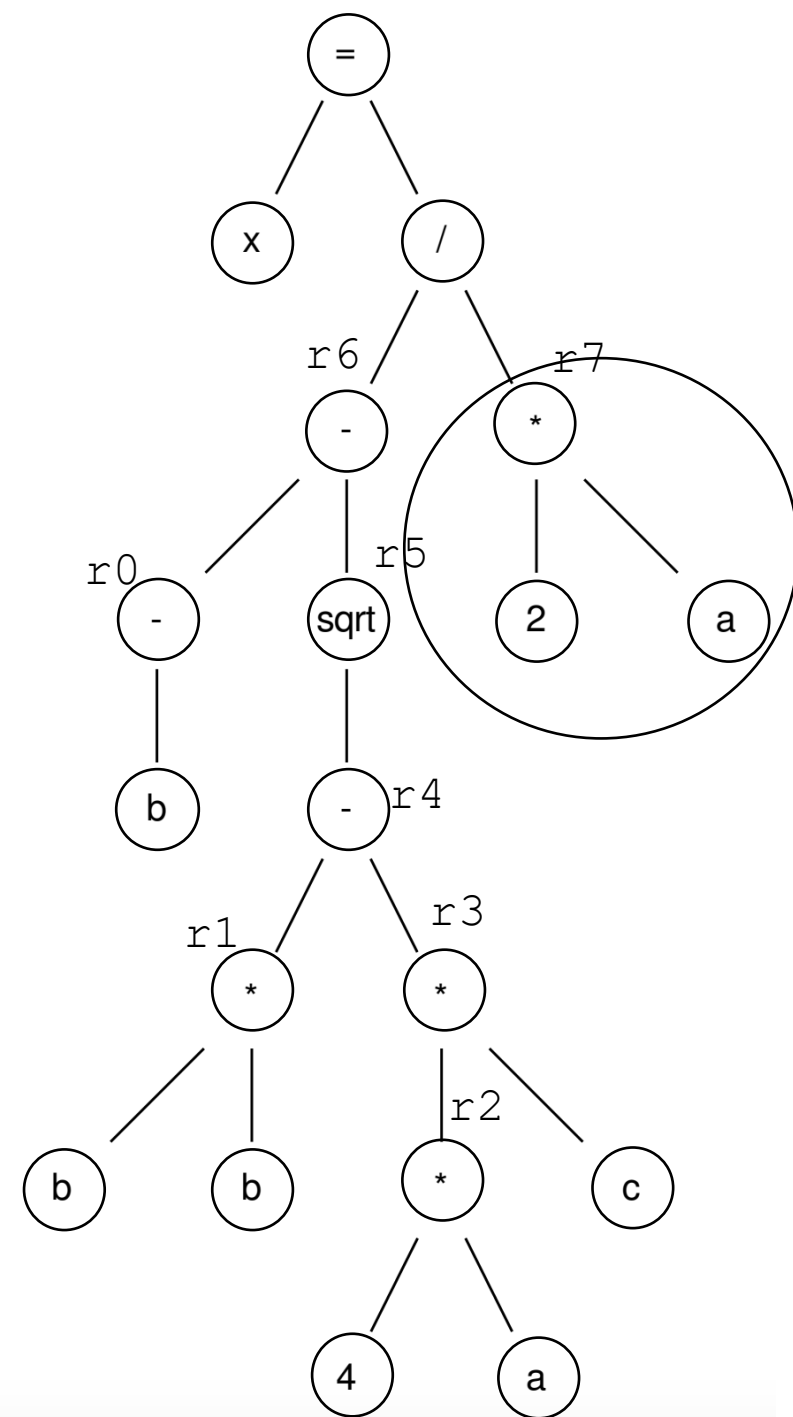
```



Simplify this code:

post-order traversal, using temporary variables

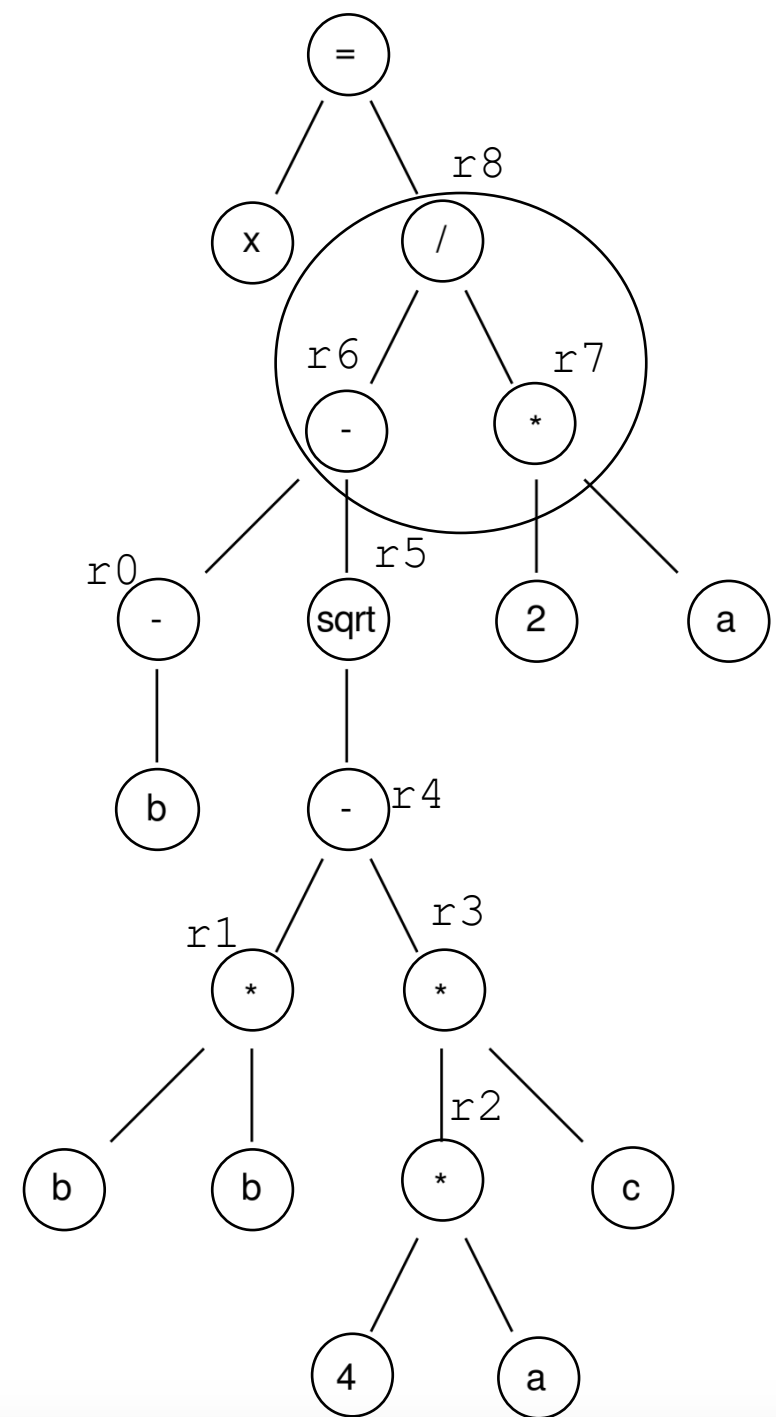
```
r0 = neg(b) ;  
r1 = b * b ;  
r2 = 4 * a ;  
r3 = r2 * c ;  
r4 = r1 - r3 ;  
r5 = sqrt(r4) ;  
r6 = r0 - r5 ;  
r7 = 2 * a ;
```



Simplify this code:

post-order traversal, using temporary variables

```
r0 = neg(b) ;  
r1 = b * b ;  
r2 = 4 * a ;  
r3 = r2 * c ;  
r4 = r1 - r3 ;  
r5 = sqrt(r4) ;  
r6 = r0 - r5 ;  
r7 = 2 * a ;  
r8 = r6 / r7 ;
```



Simplify this code:

post-order traversal, using temporary variables

```
r0 = neg(b) ;  
r1 = b * b ;  
r2 = 4 * a ;  
r3 = r2 * c ;  
r4 = r1 - r3 ;  
r5 = sqrt(r4) ;  
r6 = r0 - r5 ;  
r7 = 2 * a ;  
r8 = r6 / r7 ;  
x = r8 ;
```

- This is not exactly an ISA
 - unlimited registers
 - not always a 1-1 mapping of instructions.
- but it is much easier to translate to the ISA
- We call this an intermediate representation, or IR
- Examples of IR:
LLVM, SPIR-V (IR for GPUs)

```
1 // Type your code here, or load an example.
```

```
2 float sqrt(float x);
```

```
3
```

```
4 float add(float a, float b, float c) {
```

```
5     return (-b - sqrt(b*b - 4 * a * c)) / (2*a);
```

```
6
```

```
7 }
```

```
8
```

C program

A ▾

⚙️ Output... ▾

🔍 Filter... ▾

📖 Libraries

+ Add new... ▾

🔧 Add tool... ▾

```
1
```

```
2 define dso_local float @_Z3addfff(float %0, float %1, float %2) #0 !dbg !
```

```
3     %4 = alloca float, align 4
```

```
4     %5 = alloca float, align 4
```

```
5     %6 = alloca float, align 4
```

```
6     store float %0, float* %4, align 4
```

```
7     call void @llvm.dbg.declare(metadata float* %4, metadata !12, metadata
```

```
8     store float %1, float* %5, align 4
```

```
9     call void @llvm.dbg.declare(metadata float* %5, metadata !14, metadata
```

```
10    store float %2, float* %6, align 4
```

```
11    call void @llvm.dbg.declare(metadata float* %6, metadata !16, metadata
```

```
12    %7 = load float, float* %5, align 4, !dbg !18
```

```
13    %8 = fneg float %7, !dbg !19
```

```
14    %9 = load float, float* %5, align 4, !dbg !20
```

```
15    %10 = load float, float* %5, align 4, !dbg !21
```

```
16    %11 = fmul float %9, %10, !dbg !22
```

```
17    %12 = load float, float* %4, align 4, !dbg !23
```

```
18    %13 = fmul float 4.000000e+00, %12, !dbg !24
```

```
19    %14 = load float, float* %6, align 4, !dbg !25
```

```
20    %15 = fmul float %13, %14, !dbg !26
```

```
21    %16 = fsub float %11, %15, !dbg !27
```

```
22    %17 = call float @_Z4sqrtf(float %16), !dbg !28
```

```
23    %18 = fsub float %8, %17, !dbg !29
```

```
24    %19 = load float, float* %4, align 4, !dbg !30
```

```
25    %20 = fmul float 2.000000e+00, %19, !dbg !31
```

```
26    %21 = fdiv float %18, %20, !dbg !32
```

```
27    ret float %21, !dbg !33
```

```
28 }
```

llvm IR

Memory accesses

```
int increment(int *a) {  
    a[0]++;  
}
```

```
%5 = load i32, i32* %4  
%6 = add nsw i32 %5, 1  
store i32 %6, i32* %4
```

Unless explicitly expressed in the programming language, loads and stores are split into multiple instructions!

Zoom out

- This can be a lot if you don't have a compiler background; don't feel overwhelmed!
- To be successful in this class, you don't need to be an expert on compilation, ISAs, or IRs.
- The important thing is to have a mental model of how your complex code is broken down into instructions that are executed on hardware, especially loads and stores

New material – Instruction Level Parallelism

Instruction-level Parallelism (ILP)

- Parallelism from a **single stream of instructions**.
 - Output of program must match exactly a sequential execution!
- Widely applicable:
 - most mainstream programming languages are **sequential**
 - most deployed **hardware** has components to execute ILP
- Done by a combination of **programmer, compiler, and hardware**

Instruction-level Parallelism (ILP)

- What type of instructions can be executed in parallel?

Instruction-level Parallelism (ILP)

- What type of instructions can be done in parallel?

two instructions can be executed in parallel if they are independent

Instruction-level Parallelism (ILP)

- What type of instructions can be done in parallel?

*two instructions can be executed in parallel if they are **independent***

```
x = z + w;  
a = b + c;
```

*Two instructions are independent if the **operand registers** are disjoint from the **result registers***

(assume all letter variables are registers)

Instruction-level Parallelism (ILP)

- What type of instructions can be done in parallel?

two instructions can be executed in parallel if they are independent

```
x = z + w;  
a = b + c;
```

Two instructions are independent if the operand registers are disjoint from the result registers

(assume all letter variables are registers)

instructions that are not independent cannot be executed in parallel

```
x = z + w;  
a = b + x;
```

Instruction-level Parallelism (ILP)

- What type of instructions can be done in parallel?

two instructions can be executed in parallel if they are independent

```
x = z + w;  
a = b + c;
```

Two instructions are independent if the operand registers are disjoint from the result registers

(assume all letter variables are registers)

instructions that are not independent cannot be executed in parallel

```
x = z + w;  
a = b + x;
```

*Many times, dependencies can be easily tracked in the **compiler**:*

How can hardware execute ILP?

- Executing multiple instructions at once:
- Very Long Instruction Word (VLIW) architecture
 - Complicated architecture
 - Compiler combines multiple instructions into one
- Superscalar architecture:
 - Several sequential operations are issued in parallel
- Pipeline architecture:
 - Send instructions one after the other through stages

How can hardware execute ILP?

- Superscalar architecture:
 - Several sequential operations are issued in parallel
 - hardware detects dependencies

```
instr0;  
instr1;  
instr2;
```

***issue-width** is maximum number of instructions that can be issued in parallel*

How can hardware execute ILP?

- Superscalar architecture:
 - Several sequential operations are issued in parallel
 - hardware detects dependencies

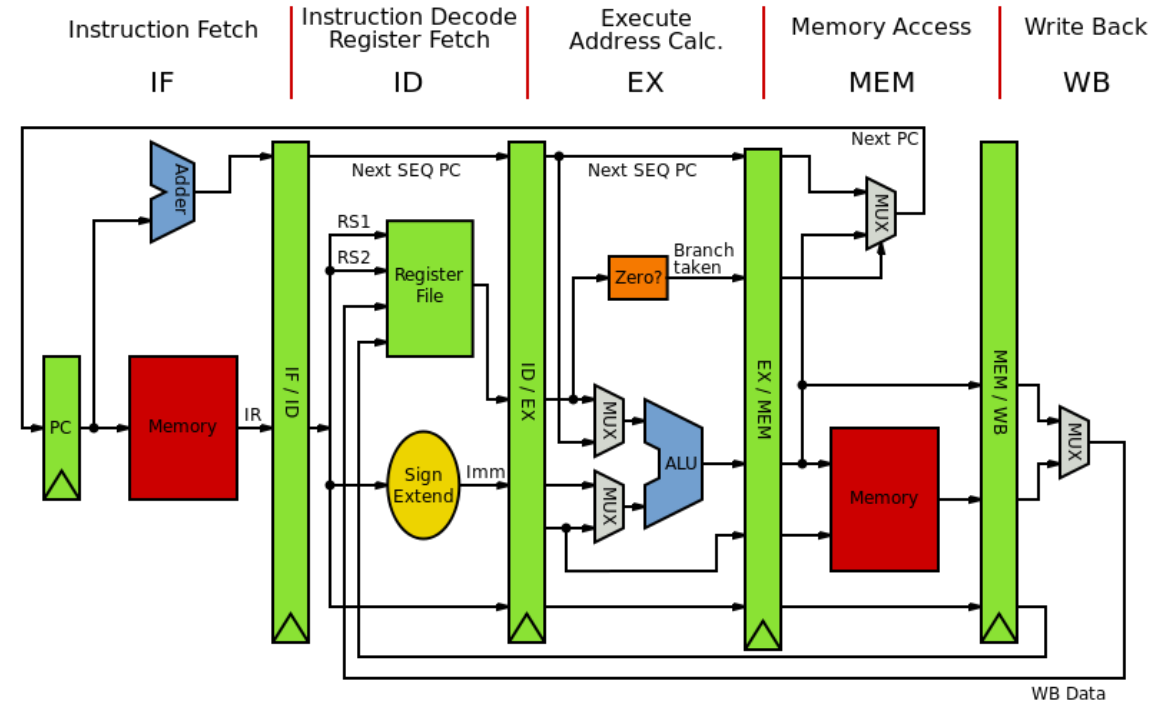
```
instr0;  
instr1;  
instr2;
```

***issue-width** is maximum number of instructions that can be issued in parallel*

if instr0 and instr1 are independent, they will be issued in parallel

How can hardware execute ILP?

- Pipeline parallelism
- Abstract mental model:
 - N-stage **pipeline**
 - N instructions can be in-flight
 - **Dependencies stall** pipeline



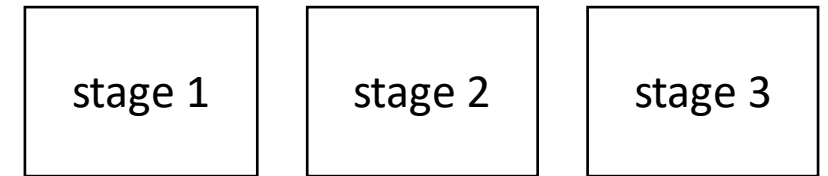
MIPS pipeline image from:

[https://commons.wikimedia.org/wiki/Pipeline_\(computer_hardware\)](https://commons.wikimedia.org/wiki/Pipeline_(computer_hardware))

Pipeline

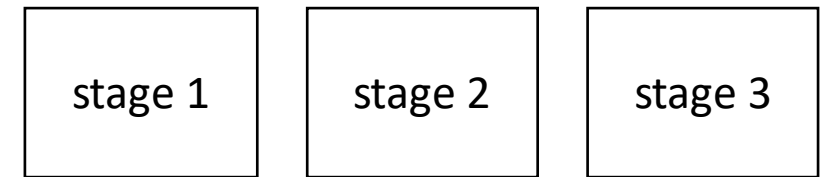
- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline

```
instr1;  
instr2;  
instr3;
```



Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline

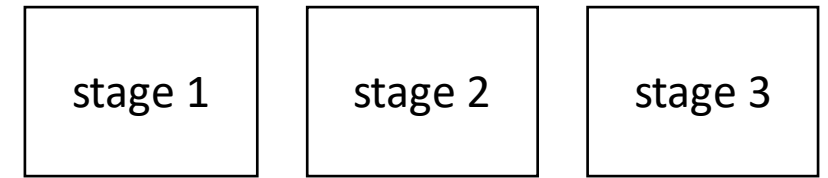


`instr1;`

`instr2;`
`instr3;`

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline



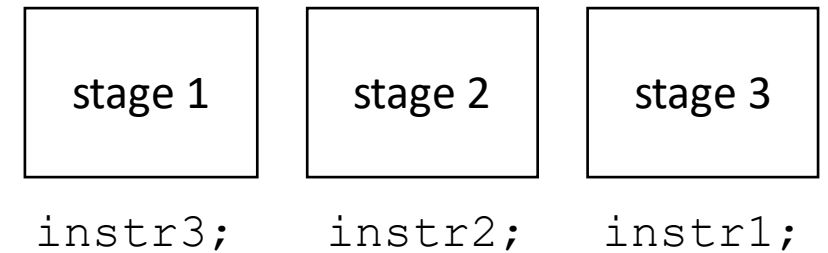
`instr2;`

`instr1;`

`instr3;`

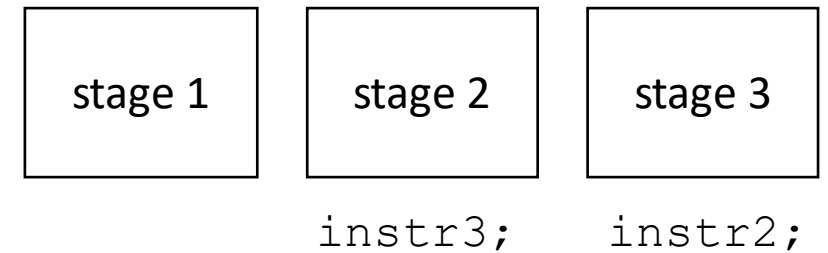
Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline



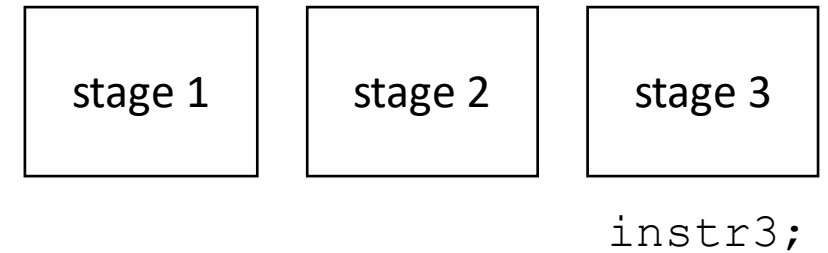
Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline



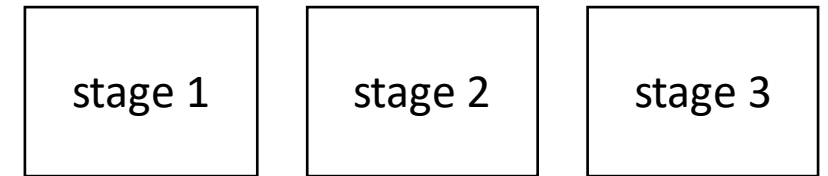
Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline



Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline

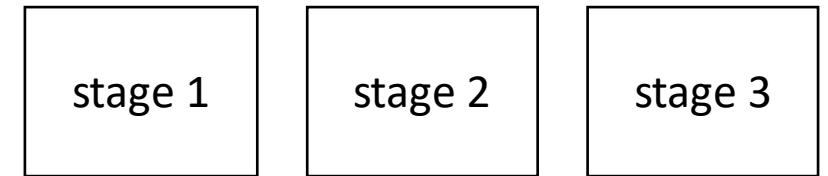


6 cycles for 3 independent instructions

Converges 1 instruction per cycle

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline

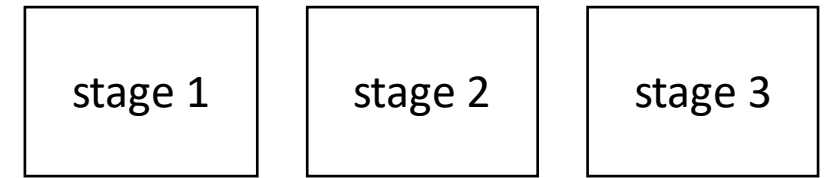


```
instr1;  
instr2;  
instr3;
```

*What if the
instructions depend on
each other?*

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline



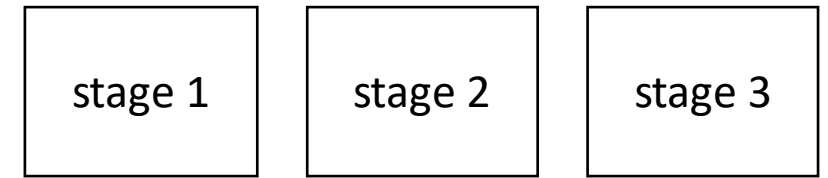
`instr1;`

`instr2;`
`instr3;`

*What if the
instructions depend on
each other?*

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline



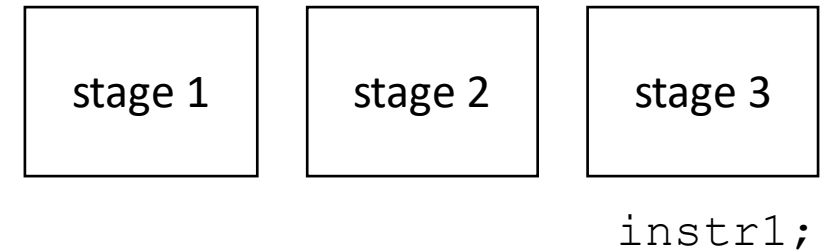
`instr1;`

`instr2;`
`instr3;`

*What if the
instructions depend on
each other?*

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline

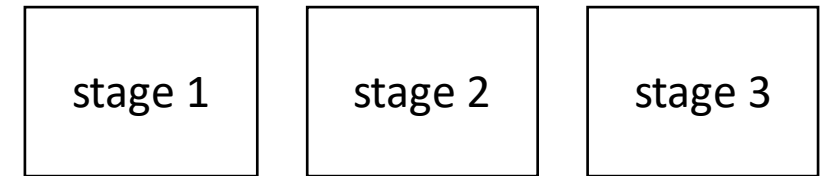


```
instr2;  
instr3;
```

*What if the
instructions depend on
each other?*

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline

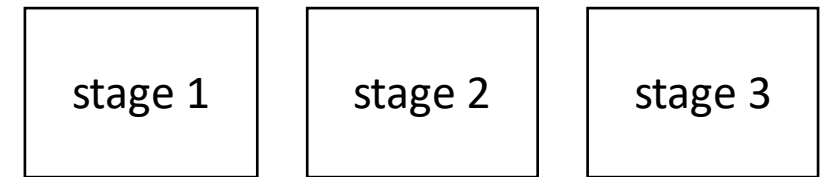


```
instr2;  
instr3;
```

*What if the
instructions depend on
each other?*

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline



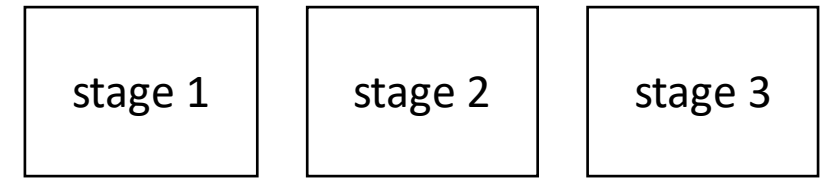
`instr2;`

`instr3;`

*What if the
instructions depend on
each other?*

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline



`instr2;`

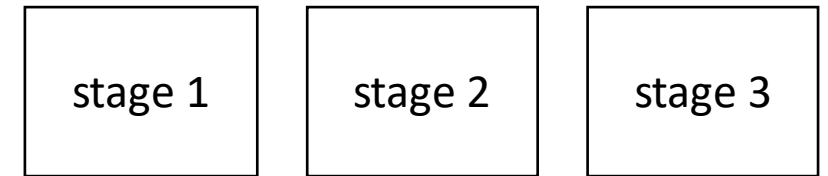
`instr3;`

and so on...

*What if the
instructions depend on
each other?*

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline



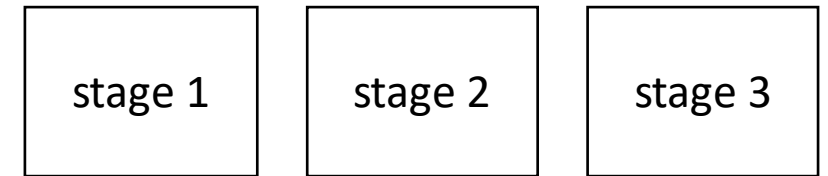
*What if the
instructions depend on
each other?*

9 cycles for 3 instructions

converges to 3 cycles per
instruction

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline



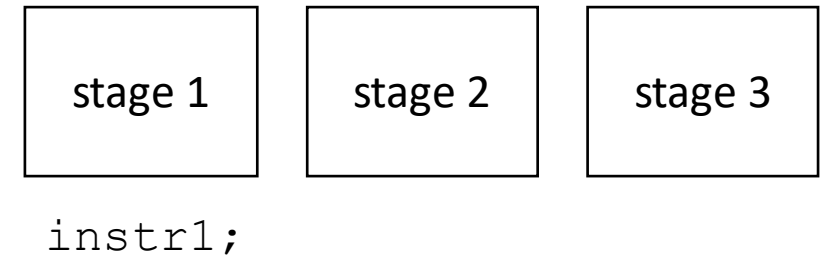
```
instr1;  
instrX0;  
instrX1;  
instr2;  
instrX2;  
instrX3;  
instr3;
```

If there are non-dependent instructions from other places in the program that we can interleave then we can get back performance!

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline

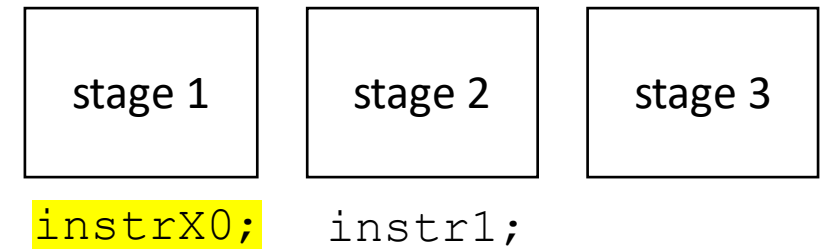
```
instrX0;  
instrX1;  
instr2;  
instrX2;  
instrX3;  
instr3;
```



If there are non-dependent instructions from other places in the program that we can interleave then we can get back performance!

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline

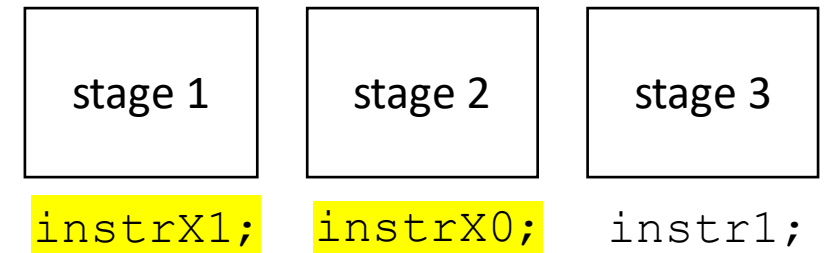


```
instrX1;  
instr2;  
instrX2;  
instrX3;  
instr3;
```

If there are non-dependent instructions from other places in the program that we can interleave then we can get back performance!

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline

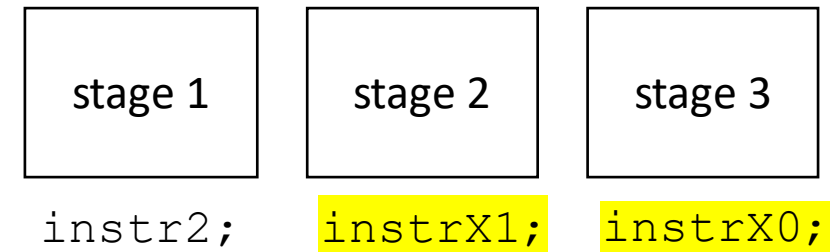


```
instr2;  
instrX2;  
instrX3;  
instr3;
```

If there are non-dependent instructions from other places in the program that we can interleave then we can get back performance!

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline

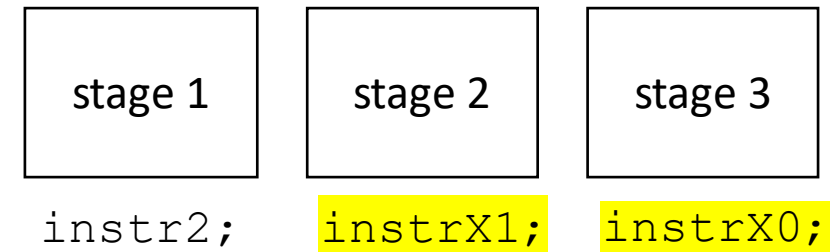


instrX2;
instrX3;
instr3;

If there are non-dependent instructions from other places in the program that we can interleave then we can get back performance!

Pipeline

- Pipeline parallelism
- Abstract mental model for compiler:
 - N-stage pipeline
 - N instructions can be in-flight
 - Dependencies stall pipeline



instrX2;
instrX3;
instr3;

and so on...

We converge to 1 cycle per instruction again!

If there are non-dependent instructions from other places in the program that we can interleave then we can get back performance!

Independent Instructions

- Out-of-order execution
 - Hardware looks ahead for independent instructions
 - Hardware delays dependent instructions

What does this look like in the real world?

- Intel Haswell (2013):
 - Issue width of 4
 - 14-19 stage pipeline
 - OoO execution
- Intel Nehalem (2008)
 - Issue width of 2-4
 - 20-24 stage pipeline
 - OoO execution
- ARM
 - Cortex V8 has issue width of 2
 - V7 has 3 stage pipeline; Cortex V8 has 13
 - OoO execution
- RISC-V
 - Ariane and Rocket (open-source processor implementations) are In-Order
 - 3-6 stage pipelines
 - some superscalar implementations (BOOM)

What does this mean for us?

- We should have an abstract performance model for instruction scheduling (the order of instructions)
- Try not to place dependent instructions in sequence
- The compiler will often help us here, but sometimes it cannot!

Two techniques to optimize for ILP

- Independent for loops (loop unrolling)
- Reduction for loops (loop unrolling)

What is loop unrolling?

```
for (int i = 0; i < 12; i++) {  
    a[i] = b[i] + c[i];  
}
```

Can we unroll this loop?
Data and **control** dependencies

```
for (int i = 0; i < 12; i+=2) {  
    a[i] = b[i] + c[i];  
    a[i+1] = b[i+1] + c[i+1];  
}
```

Using Loop Unrolling to Exploit ILP

- for loops with independent chains of computation

```
for (int i = 0; i < SIZE; i++) {  
    SEQ(i);  
}
```

where: $SEQ(i) = instr1;$
 $instr2;$

and let $instr(N)$ depend on $instr(N-1)$

...
 $a[i] = instrN;$

loops only write to memory
addressed by the loop variable

Using Loop Unrolling to Exploit ILP

- Simple loop unrolling:

```
for (int i = 0; i < SIZE; i+=2) {  
    SEQ(i);  
    SEQ(i+1);  
}
```

Saves one addition and one comparison per loop, but doesn't help with ILP

Using Loop Unrolling to Exploit ILP

- Simple loop unrolling:

```
for (int i = 0; i < SIZE; i+=2) {  
    SEQ(i);  
    SEQ(i+1);  
}
```

Let **green highlights** indicate instructions from iteration i .

Let **blue highlights** indicate instructions from iteration $i + 1$.

Using Loop Unrolling to Exploit ILP

- Simple loop unrolling:

```
for (int i = 0; i < SIZE; i+=2) {  
    SEQ(i);  
    SEQ(i+1);  
}
```

Let $SEQ(i, j)$ be the j th instruction of $SEQ(i)$.

Let each instruction chain have N instructions

Using Loop Unrolling to Exploit ILP

- Simple loop unrolling:

```
for (int i = 0; i < SIZE; i+=2) {  
    SEQ(i, 1);  
    SEQ(i, 2);  
    ...  
    SEQ(i, N); // end iteration for i  
    SEQ(i+1, 1);  
    SEQ(i+1, 2);  
    ...  
    SEQ(i+1, N); // end iteration for i + 1  
}
```

Let $SEQ(i, j)$ be the j th instruction of $SEQ(i)$.

Let each instruction chain have N instructions

Using Loop Unrolling to Exploit ILP

- Simple loop unrolling:

```
for (int i = 0; i < SIZE; i+=2) {  
    SEQ(i, 1);  
    SEQ(i+1, 1);  
    SEQ(i, 2);  
    SEQ(i+1, 2);  
    ...  
    SEQ(i, N);  
    SEQ(i+1, N);  
}
```

They can be interleaved

Using Loop Unrolling to Exploit ILP

- Simple loop unrolling:

```
for (int i = 0; i < SIZE; i+=2) {  
    SEQ(i, 1);  
    SEQ(i+1, 1);  
    SEQ(i, 2);  
    SEQ(i+1, 2);  
    ...  
    SEQ(i, N);  
    SEQ(i+1, N);  
}
```

They can be interleaved

two instructions can be pipelined, or executed
on a superscalar processor

Using Loop Unrolling to Exploit ILP

- This is what you are doing in part 1 of homework 1
- You are playing the role of a compiler unrolling loops
- Your “compiler” is written in Python. You print out C++ code
- Your code is parameterized by dependency chain and by unroll factor

Loop Unrolling for Reduction Loops

- Prior approach examined loops with independent iterations and chains of dependent computations
- Now we will look at reduction loops:
 - Entire computation is dependent
 - Typically, short bodies (addition, multiplication, max, min)

1	2	3	4	5	6
---	---	---	---	---	---

addition: 21

max: 6

min: 1

Loop Unrolling for Reduction Loops

- Simple implementation:

```
for (int i = 1; i < SIZE; i++) {  
    a[0] = REDUCE(a[0], a[i]);  
}
```

1	2	3	4	5	6
---	---	---	---	---	---

1 + 2 + 3 + 4 + 5 + 6

Loop Unrolling for Reduction Loops

- Simple implementation:

```
for (int i = 1; i < SIZE; i++) {  
    a[0] = REDUCE(a[0], a[i]);  
}
```

What is associativity?

1	2	3	4	5	6
---	---	---	---	---	---

$(1 + 2 + 3) + (4 + 5 + 6)$

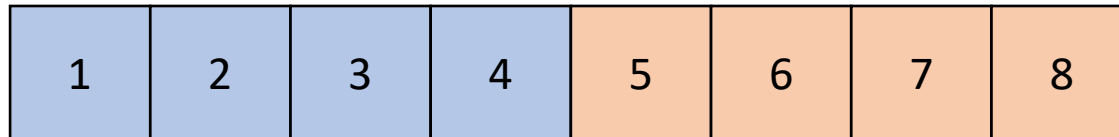
Loop Unrolling for Reduction Loops

- Chunk array in equal sized partitions and do local reductions
- Consider size 2:

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

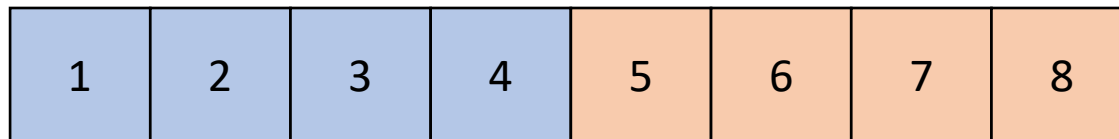
Loop Unrolling for Reduction Loops

- chunk array in equal sized partitions and do local reductions
- Consider size 2:



Loop Unrolling for Reduction Loops

- chunk array in equal sized partitions and do local reductions
- Consider size 2:



Do addition reduction in base memory location

Loop Unrolling for Reduction Loops

- chunk array in equal sized partitions and do local reductions
- Consider size 2:

10	2	3	4	26	6	7	8
----	---	---	---	----	---	---	---

Do addition reduction in base memory location

Loop Unrolling for Reduction Loops

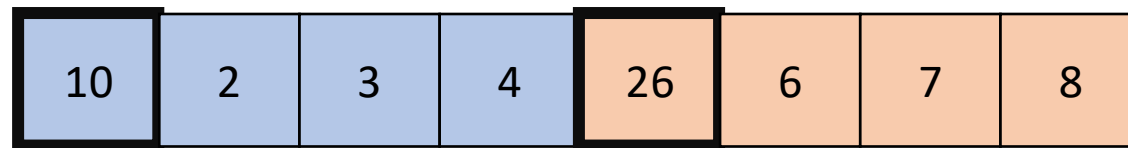
- chunk array in equal sized partitions and do local reductions
- Consider size 2:

10	2	3	4	26	6	7	8
----	---	---	---	----	---	---	---

Add together base locations

Loop Unrolling for Reduction Loops

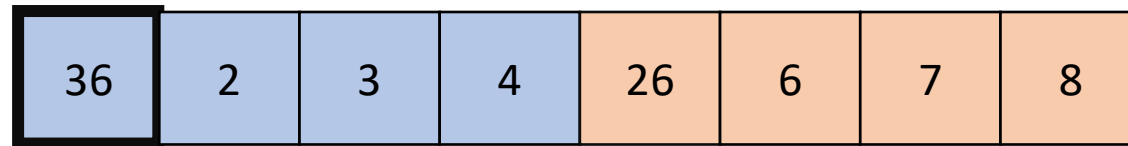
- chunk array in equal sized partitions and do local reductions
- Consider size 2:



Add together base locations

Loop Unrolling for Reduction Loops

- chunk array in equal sized partitions and do local reductions
- Consider size 2:



Add together base locations

Loop Unrolling for Reduction Loops

- Simple implementation:

```
for (int i = 1; i < SIZE/2; i++) {  
    a[0] = REDUCE(a[0], a[i]);  
    a[SIZE/2] = REDUCE(a[SIZE/2], a[(SIZE/2)+i]);  
}
```

```
a[0] = REDUCE(a[0], a[SIZE/2])
```

Loop Unrolling for Reduction Loops

- Simple implementation:

```
for (int i = 1; i < SIZE/2; i++) {  
    a[0] = REDUCE(a[0], a[i]);  
    a[SIZE/2] = REDUCE(a[SIZE/2], a[(SIZE/2)+i]);  
}
```

```
a[0] = REDUCE(a[0], a[SIZE/2])
```

Loop Unrolling for Reduction Loops

- Simple implementation:

```
for (int i = 1; i < SIZE/2; i++) {  
    a[0] = REDUCE(a[0], a[i]);  
    a[SIZE/2] = REDUCE(a[SIZE/2], a[(SIZE/2)+i]);  
}
```

```
a[0] = REDUCE(a[0], a[SIZE/2])
```

*independent
instructions
can be done
in parallel!*

Watch out!

- Our abstraction: separate dependent instructions as far as possible
- Pros:
 - Simple
- Cons:
 - Can lead to register spilling, causing expensive loads

consider `instr1` and `instr2` have a data dependence, and `instrX`'s are independent

```
instr1;
```

```
instrX0; | independent instructions. If they overwrite the register storing instr1's result, then it will have to  
instrX1; | be stored to memory and retrieved before instr2
```

```
...
```

```
Instr2;
```

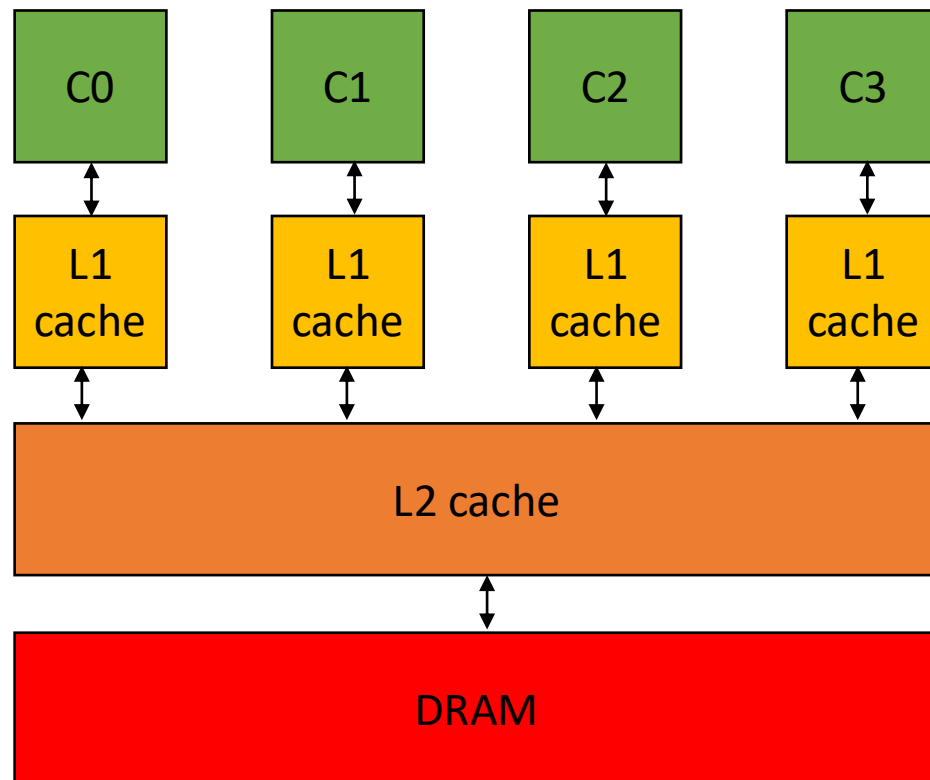
Watch out!

- Our abstraction: separate dependent instructions as far as possible
- Pros:
 - Simple
- Cons:
 - Can lead to register spilling, causing expensive loads

Solutions include using a **resource model** to guide the topological ordering. Highly architecture dependent. Compiler algorithms become more expensive.

Consider timing the compile time in your homework assignment.

Memory hierarchy overview



Core

A core executes a stream
of sequential ISA instructions

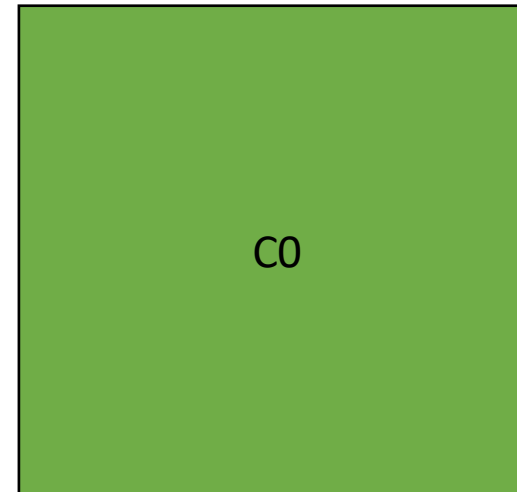
A good mental model executes
1 ISA instruction per cycle

3 Ghz means 3B cycles per second
1 ISA instruction takes .33 ns

Compiled function #0

```
13      movd    eax, xmm0
14      xor     eax, 2147483648
15      movd    xmm0, eax
16      movss   dword ptr [rbp - 16], xmm0
17      movss   xmm0, dword ptr [rbp - 8]
18      mulss   xmm0, dword ptr [rbp - 8]
19      movss   xmm1, dword ptr [rip + .LCPI0_1]
20      mulss   xmm1, dword ptr [rbp - 4]
21      mulss   xmm1, dword ptr [rbp - 12]
22      subss   xmm0, xmm1
23      call    sqrt(float)
24      movaps  xmm1, xmm0
25      movss   xmm0, dword ptr [rbp - 16]
26      subss   xmm0, xmm1
27      movss   xmm1, dword ptr [rip + .LCPI0_0]
28      mulss   xmm1, dword ptr [rbp - 4]
29      divss   xmm0, xmm1
```

Thread 0



Core

Core

Sometimes multiple programs want to share the same core.

Compiled function #0

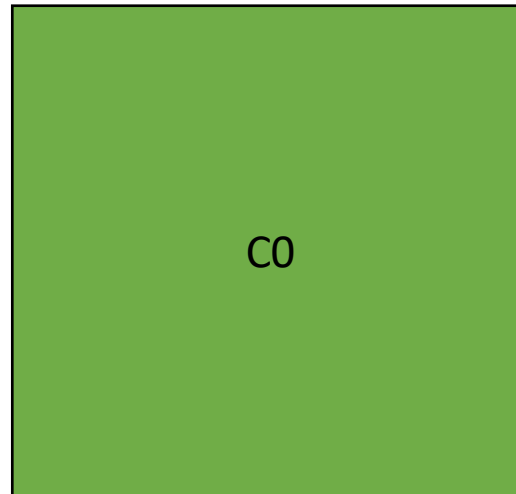
```
13      movd    eax, xmm0
14      xor     eax, 2147483648
15      movd    xmm0, eax
16      movss   dword ptr [rbp - 16], xmm0
17      movss   xmm0, dword ptr [rbp - 8]
18      mulss   xmm0, dword ptr [rbp - 8]
19      movss   xmm1, dword ptr [rip + .LCPI0_1]
20      mulss   xmm1, dword ptr [rbp - 4]
21      mulss   xmm1, dword ptr [rbp - 12]
22      subss   xmm0, xmm1
23      call    sqrt(float)
24      movaps  xmm1, xmm0
25      movss   xmm0, dword ptr [rbp - 16]
26      subss   xmm0, xmm1
27      movss   xmm1, dword ptr [rip + .LCPI0_0]
28      mulss   xmm1, dword ptr [rbp - 4]
29      divss   xmm0, xmm1
```

Thread 0

Compiled function #1

```
movss   xmm0, dword ptr [rbp - 16]
movss   xmm0, dword ptr [rbp - 8]
mulss   xmm0, dword ptr [rbp - 8]
movss   xmm1, dword ptr [rip + .LCPI0_1]
mulss   xmm1, dword ptr [rbp - 4]
mulss   xmm1, dword ptr [rbp - 12]
subss   xmm0, xmm1
call    sqrt(float)
movaps  xmm1, xmm0
movss   xmm0, dword ptr [rbp - 16]
subss   xmm0, xmm1
movss   xmm1, dword ptr [rip + .LCPI0_0]
mulss   xmm1, dword ptr [rbp - 4]
divss   xmm0, xmm1
add     rsp, 16
```

Thread 1



Core

Core

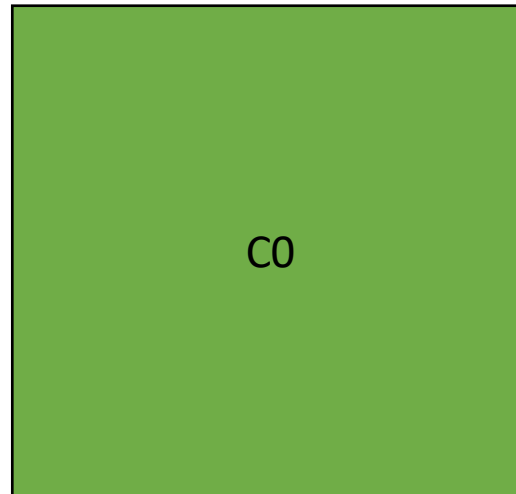
Sometimes multiple programs want to share the same core.

Compiled function #0

```
13    movd    eax, xmm0
14    xor     eax, 2147483648
15    movd    xmm0, eax
16    movss   dword ptr [rbp - 16], xmm0
17    movss   xmm0, dword ptr [rbp - 8]
18    mulss   xmm0, dword ptr [rbp - 8]
19    movss   xmm1, dword ptr [rip + .LCPI0_1]
20    mulss   xmm1, dword ptr [rbp - 4]
21    mulss   xmm1, dword ptr [rbp - 12]
22    subss   xmm0, xmm1
23    call    sqrt(float)
24    movaps  xmm1, xmm0
25    movss   xmm0, dword ptr [rbp - 16]
26    subss   xmm0, xmm1
27    movss   xmm1, dword ptr [rip + .LCPI0_0]
28    mulss   xmm1, dword ptr [rbp - 4]
29    divss   xmm0, xmm1
```



Thread 0



Core

Compiled function #1

```
movss   xmm0, dword ptr [rbp - 16]
movss   xmm0, dword ptr [rbp - 8]
mulss   xmm0, dword ptr [rbp - 8]
movss   xmm1, dword ptr [rip + .LCPI0_1]
mulss   xmm1, dword ptr [rbp - 4]
mulss   xmm1, dword ptr [rbp - 12]
subss   xmm0, xmm1
call    sqrt(float)
movaps  xmm1, xmm0
movss   xmm0, dword ptr [rbp - 16]
subss   xmm0, xmm1
movss   xmm1, dword ptr [rip + .LCPI0_0]
mulss   xmm1, dword ptr [rbp - 4]
divss   xmm0, xmm1
add     rsp, 16
```

Thread 1



The OS can preempt a thread
(remove it from the hardware resource)

Core

Compiled function #1

```
movss xmm0, dword ptr [rbp - 8]
mulss xmm0, dword ptr [rbp - 8]
movss xmm1, dword ptr [rip + .LCPI0_1]
mulss xmm1, dword ptr [rbp - 4]
mulss xmm1, dword ptr [rbp - 12]
subss xmm0, xmm1
call sqrt(float)
movaps xmm1, xmm0
movss xmm0, dword ptr [rbp - 16]
subss xmm0, xmm1
movss xmm1, dword ptr [rip + .LCPI0_0]
mulss xmm1, dword ptr [rbp - 4]
divss xmm0, xmm1
add rsp, 16
```

Compiled function #0

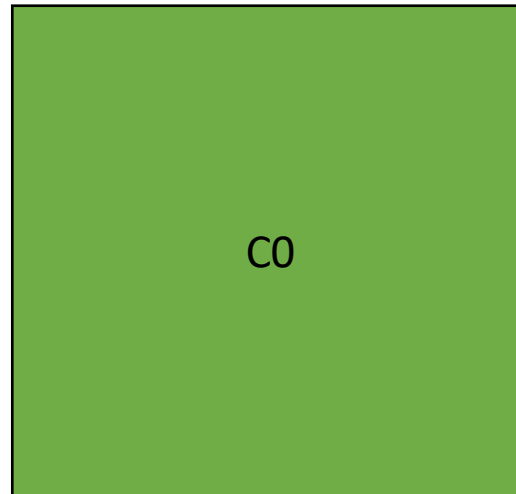
```
13 movd eax, xmm0
14 xor eax, 2147483648
15 movd xmm0, eax
16 movss dword ptr [rbp - 16], xmm0
17 movss xmm0, dword ptr [rbp - 8]
18 mulss xmm0, dword ptr [rbp - 8]
19 movss xmm1, dword ptr [rip + .LCPI0_1]
20 mulss xmm1, dword ptr [rbp - 4]
21 mulss xmm1, dword ptr [rbp - 12]
22 subss xmm0, xmm1
23 call sqrt(float)
24 movaps xmm1, xmm0
25 movss xmm0, dword ptr [rbp - 16]
26 subss xmm0, xmm1
27 movss xmm1, dword ptr [rip + .LCPI0_0]
28 mulss xmm1, dword ptr [rbp - 4]
29 divss xmm0, xmm1
```

Sometimes multiple programs want to share the same core.

This is called concurrency: multiple threads taking turns executing on the same hardware resource

Thread 1

Thread 0



Core



And place another thread to execute

Core

Preemption can occur:

- when a thread executes a long latency instruction
- periodically from the OS to provide fairness
- explicitly using sleep instructions

Compiled function #1

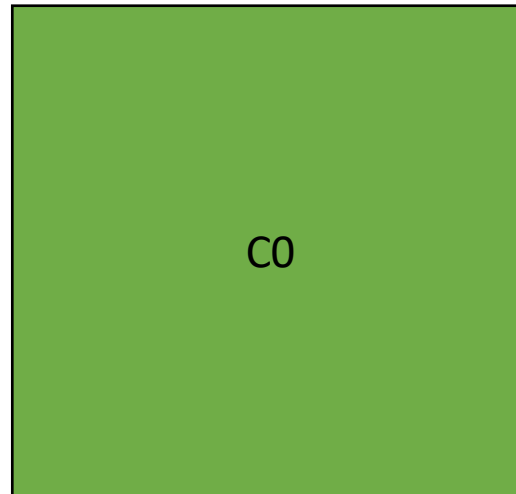
```
movss xmm0, dword ptr [rbp - 8]
mulss xmm0, dword ptr [rbp - 8]
movss xmm1, dword ptr [rip + .LCPI0_1]
mulss xmm1, dword ptr [rbp - 4]
mulss xmm1, dword ptr [rbp - 12]
subss xmm0, xmm1
call sqrt(float)
movaps xmm1, xmm0
movss xmm0, dword ptr [rbp - 16]
subss xmm0, xmm1
movss xmm1, dword ptr [rip + .LCPI0_0]
mulss xmm1, dword ptr [rbp - 4]
divss xmm0, xmm1
add rsp, 16
```

Thread 1

Compiled function #0

```
13 movd eax, xmm0
14 xor eax, 2147483648
15 movd xmm0, eax
16 movss dword ptr [rbp - 16], xmm0
17 movss xmm0, dword ptr [rbp - 8]
18 mulss xmm0, dword ptr [rbp - 8]
19 movss xmm1, dword ptr [rip + .LCPI0_1]
20 mulss xmm1, dword ptr [rbp - 4]
21 mulss xmm1, dword ptr [rbp - 12]
22 subss xmm0, xmm1
23 call sqrt(float)
24 movaps xmm1, xmm0
25 movss xmm0, dword ptr [rbp - 16]
26 subss xmm0, xmm1
27 movss xmm1, dword ptr [rip + .LCPI0_0]
28 mulss xmm1, dword ptr [rbp - 4]
29 divss xmm0, xmm1
```

Thread 0



Core



And place another thread to execute

Multicores

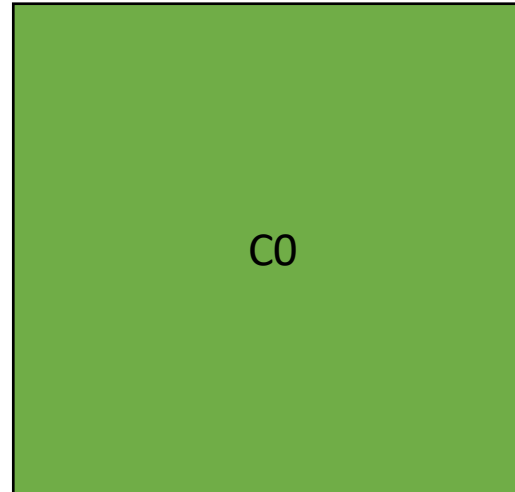
Threads can execute simultaneously (at the same time) if there are enough resources.

This is also concurrency. But when they execute at the same time, it's called: parallelism.

Compiled function #0

```
13      movd    eax, xmm0
14      xor     eax, 2147483648
15      movd    xmm0, eax
16      movss   dword ptr [rbp - 16], xmm0
17      movss   xmm0, dword ptr [rbp - 8]
18      mulss   xmm0, dword ptr [rbp - 8]
19      movss   xmm1, dword ptr [rip + .LCPI0_1]
20      mulss   xmm1, dword ptr [rbp - 4]
21      mulss   xmm1, dword ptr [rbp - 12]
22      subss   xmm0, xmm1
23      call    sqrt(float)
24      movaps  xmm1, xmm0
25      movss   xmm0, dword ptr [rbp - 16]
26      subss   xmm0, xmm1
27      movss   xmm1, dword ptr [rip + .LCPI0_0]
28      mulss   xmm1, dword ptr [rbp - 4]
29      divss   xmm0, xmm1
```

Thread 0

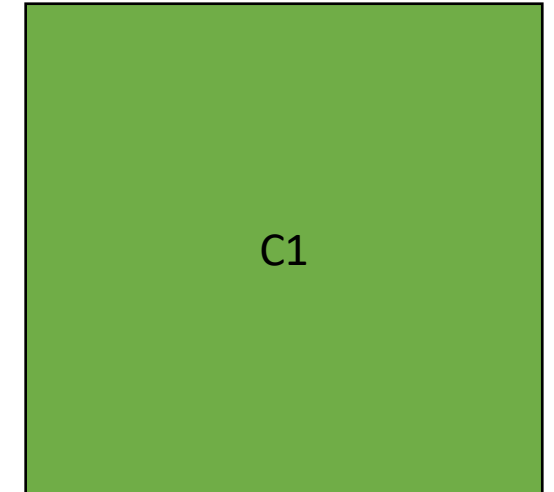


Core

Compiled function #1

```
movss   dword ptr [rbp - 16], xmm0
movss   xmm0, dword ptr [rbp - 8]
mulss   xmm0, dword ptr [rbp - 8]
movss   xmm1, dword ptr [rip + .LCPI0_1]
mulss   xmm1, dword ptr [rbp - 4]
mulss   xmm1, dword ptr [rbp - 12]
subss   xmm0, xmm1
call    sqrt(float)
movaps  xmm1, xmm0
movss   xmm0, dword ptr [rbp - 16]
subss   xmm0, xmm1
movss   xmm1, dword ptr [rip + .LCPI0_0]
mulss   xmm1, dword ptr [rbp - 4]
divss   xmm0, xmm1
add     rsp, 16
```

Thread 1



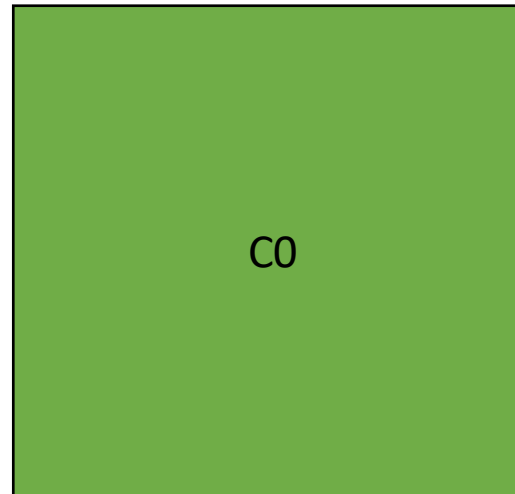
Core

Multicores

Compiled function #0

```
13      movd    eax, xmm0
14      xor     eax, 2147483648
15      movd    xmm0, eax
16      movss   dword ptr [rbp - 16], xmm0
17      movss   xmm0, dword ptr [rbp - 8]
18      mulss   xmm0, dword ptr [rbp - 8]
19      movss   xmm1, dword ptr [rip + .LCPI0_1]
20      mulss   xmm1, dword ptr [rbp - 4]
21      mulss   xmm1, dword ptr [rbp - 12]
22      subss   xmm0, xmm1
23      call    sqrt(float)
24      movaps  xmm1, xmm0
25      movss   xmm0, dword ptr [rbp - 16]
26      subss   xmm0, xmm1
27      movss   xmm1, dword ptr [rip + .LCPI0_0]
28      mulss   xmm1, dword ptr [rbp - 4]
29      divss   xmm0, xmm1
```

Thread 0

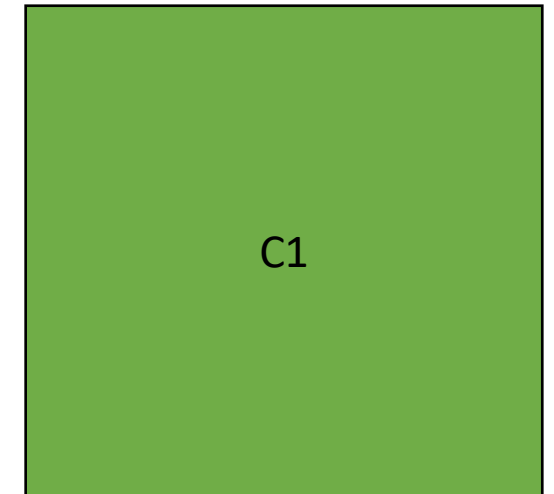


Core

Compiled function #1

```
movss   dword ptr [rbp - 16], xmm0
movss   xmm0, dword ptr [rbp - 8]
mulss   xmm0, dword ptr [rbp - 8]
movss   xmm1, dword ptr [rip + .LCPI0_1]
mulss   xmm1, dword ptr [rbp - 4]
mulss   xmm1, dword ptr [rbp - 12]
subss   xmm0, xmm1
call     sqrt(float)
movaps  xmm1, xmm0
movss   xmm0, dword ptr [rbp - 16]
subss   xmm0, xmm1
movss   xmm1, dword ptr [rip + .LCPI0_0]
mulss   xmm1, dword ptr [rbp - 4]
divss   xmm0, xmm1
add     rsp, 16
```

Thread 1



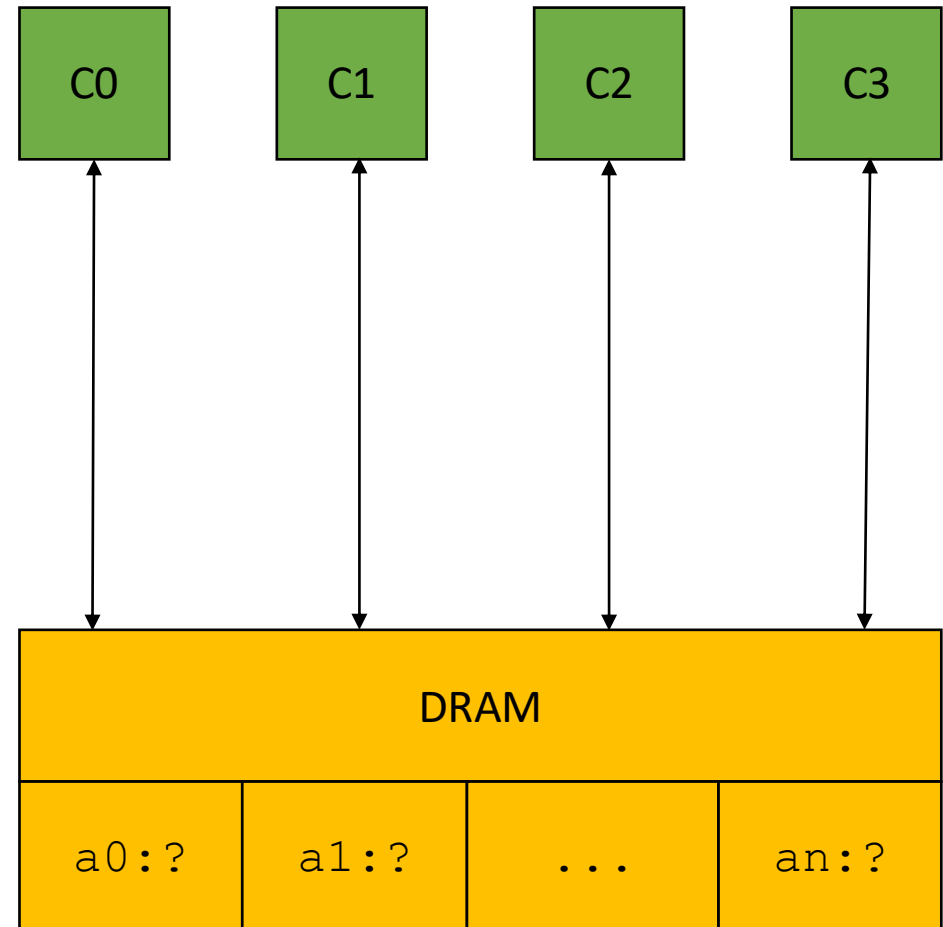
Core

This is fine if threads are independent:
e.g. running Chrome and Spotify at the
same time.

If threads need to cooperate to run
the program, then they need to communicate
through memory

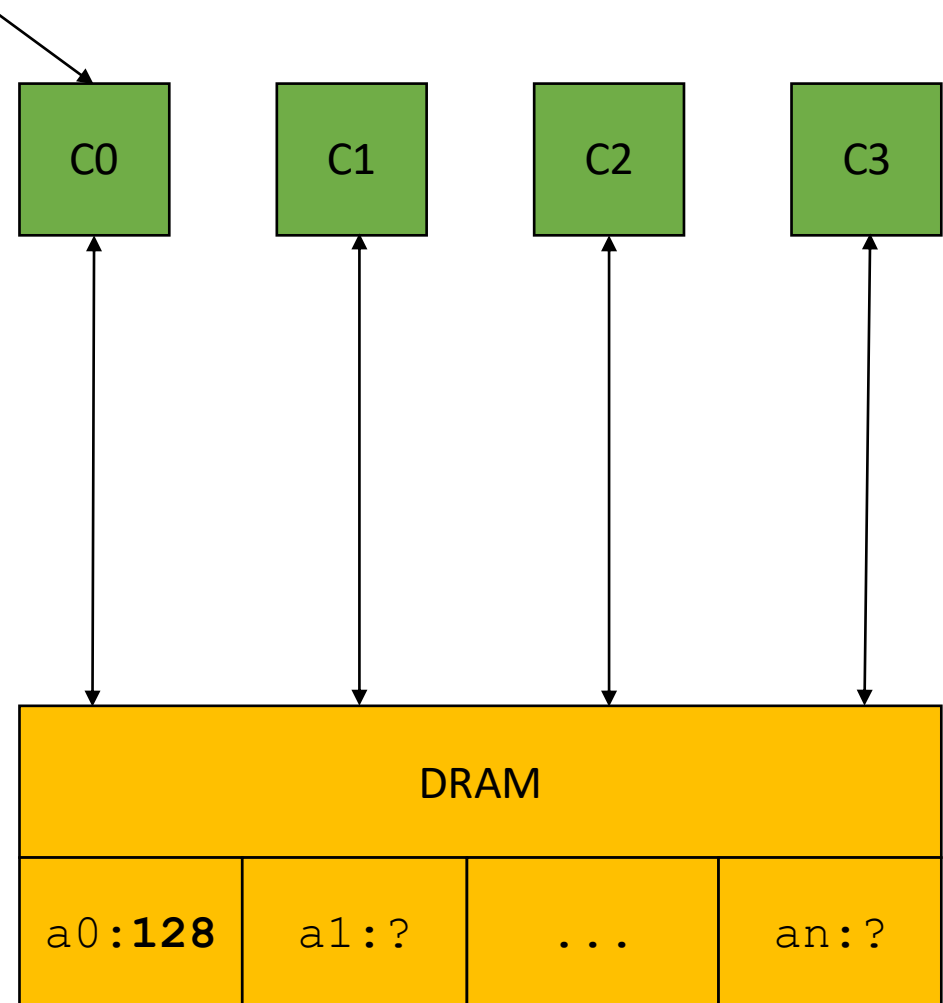
Main memory

```
store(a0, 128)
```



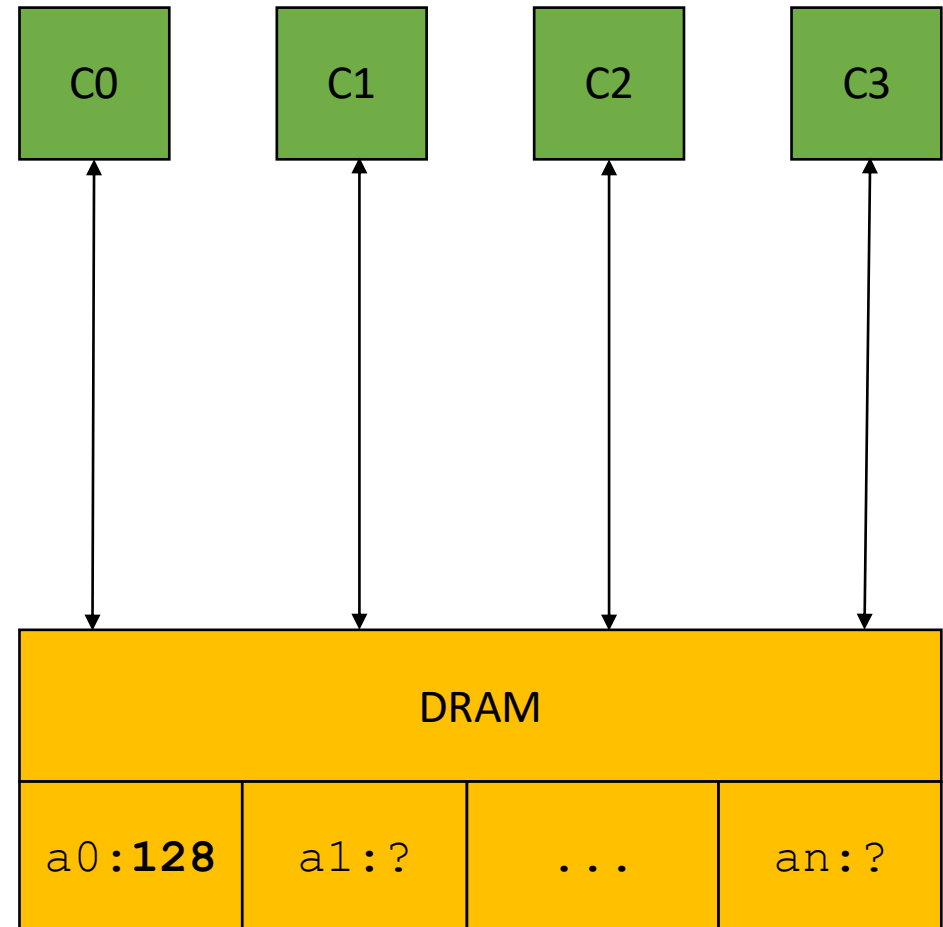
Main memory

`store(a0, 128)`

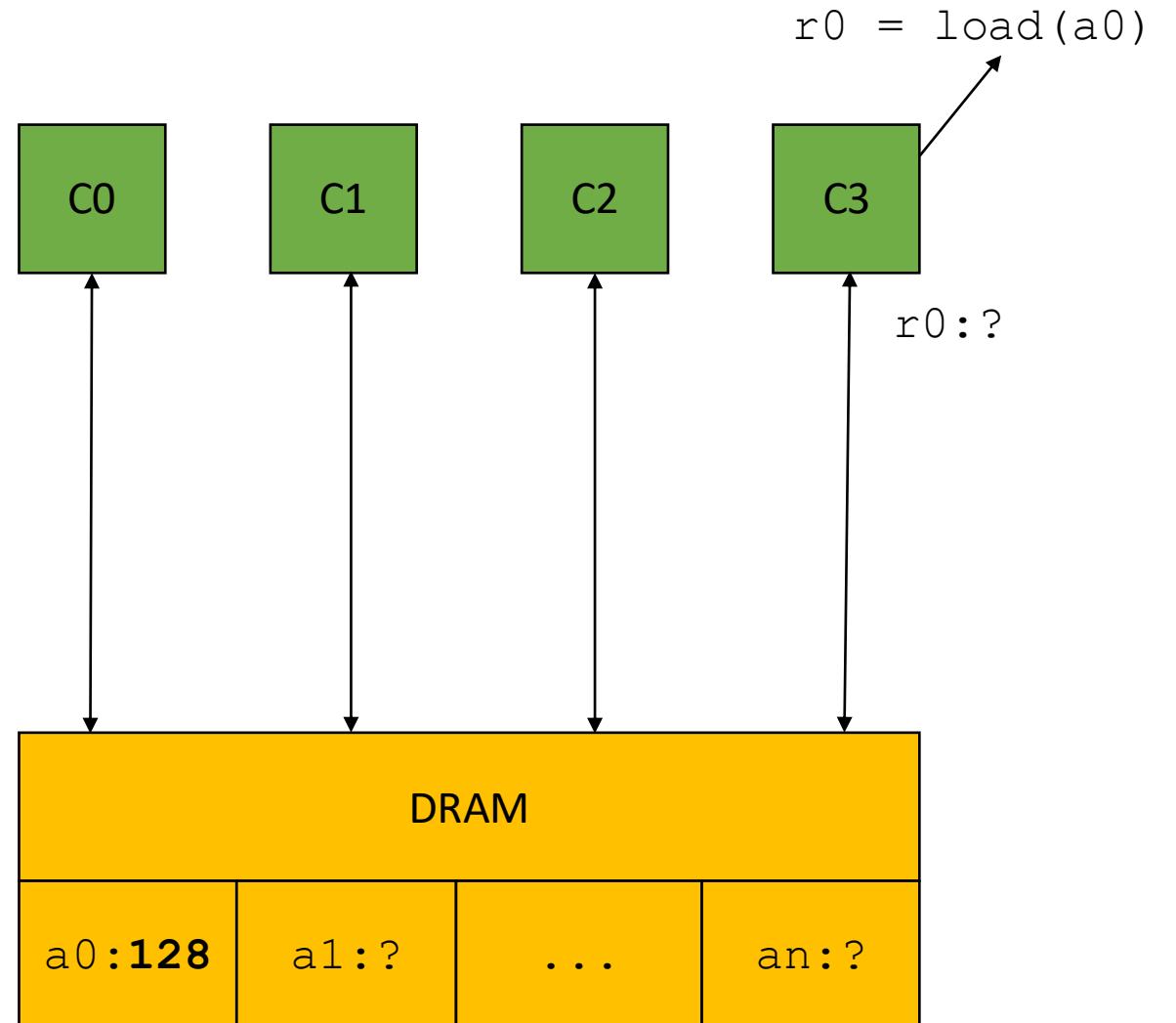


Main memory

```
r0 = load(a0)
```

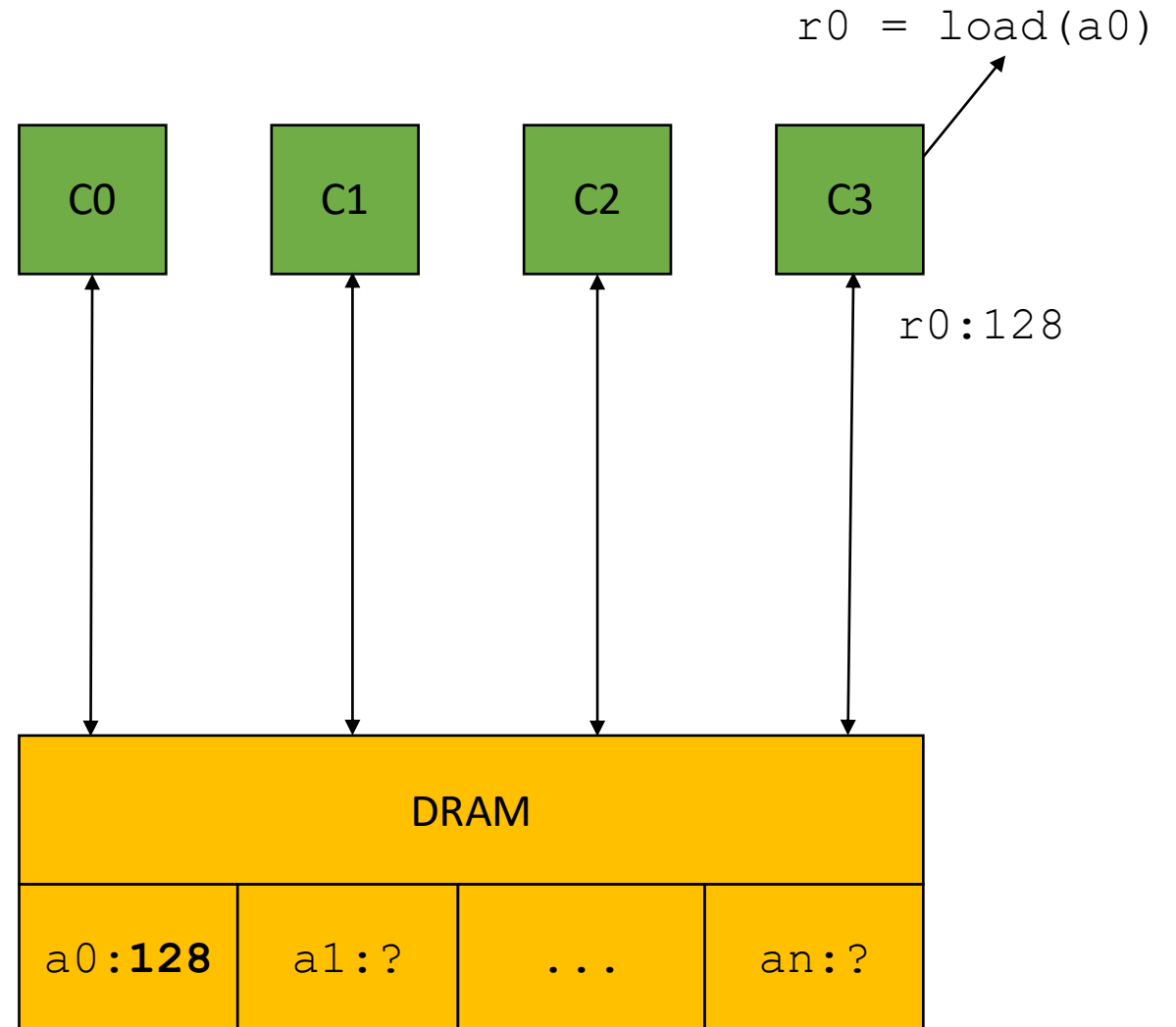


Main memory



Main memory

Problem solved!
Threads can communicate!

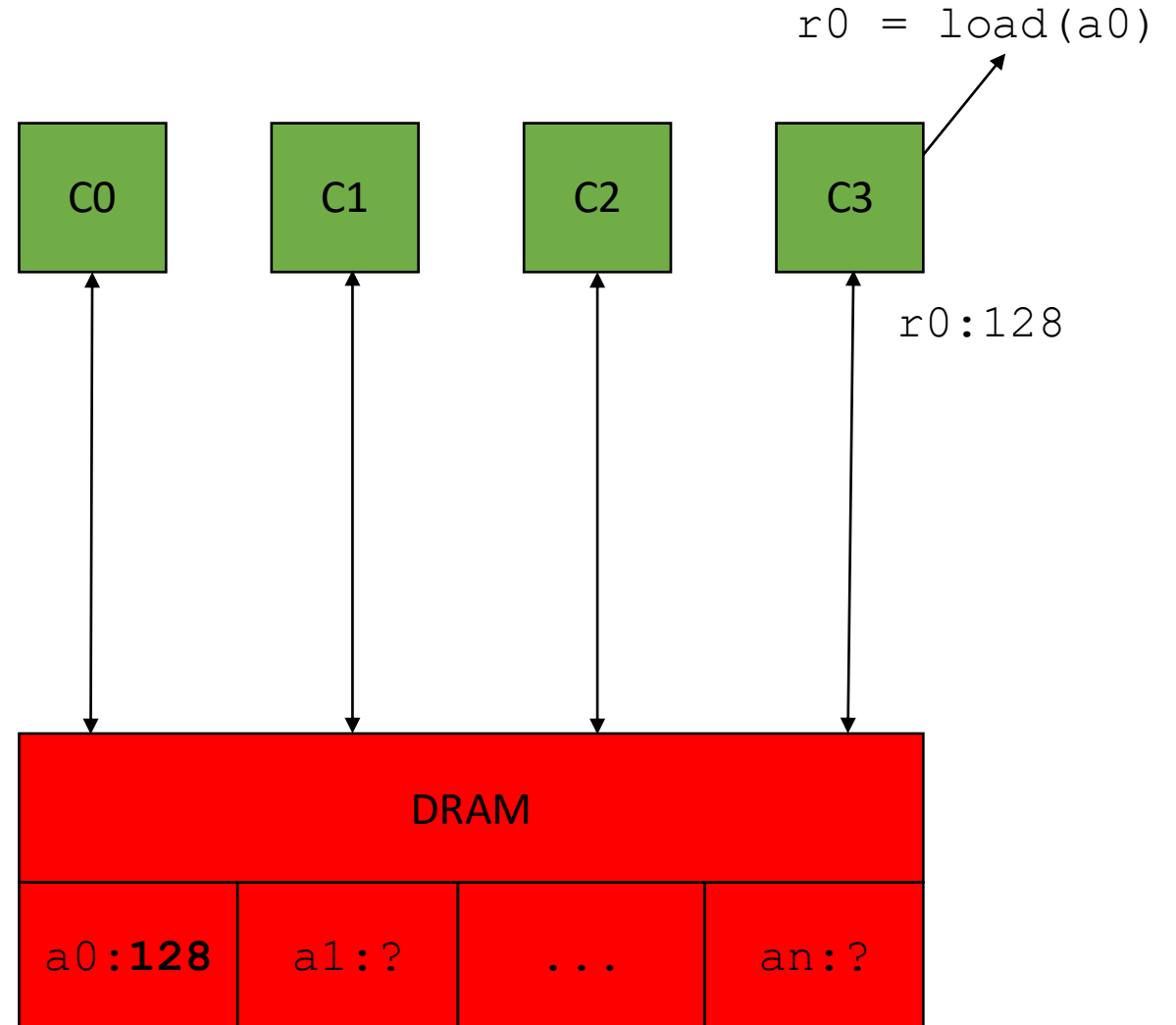


Main memory

Problem solved!

Threads can communicate!

reading a value takes ~200 cycles



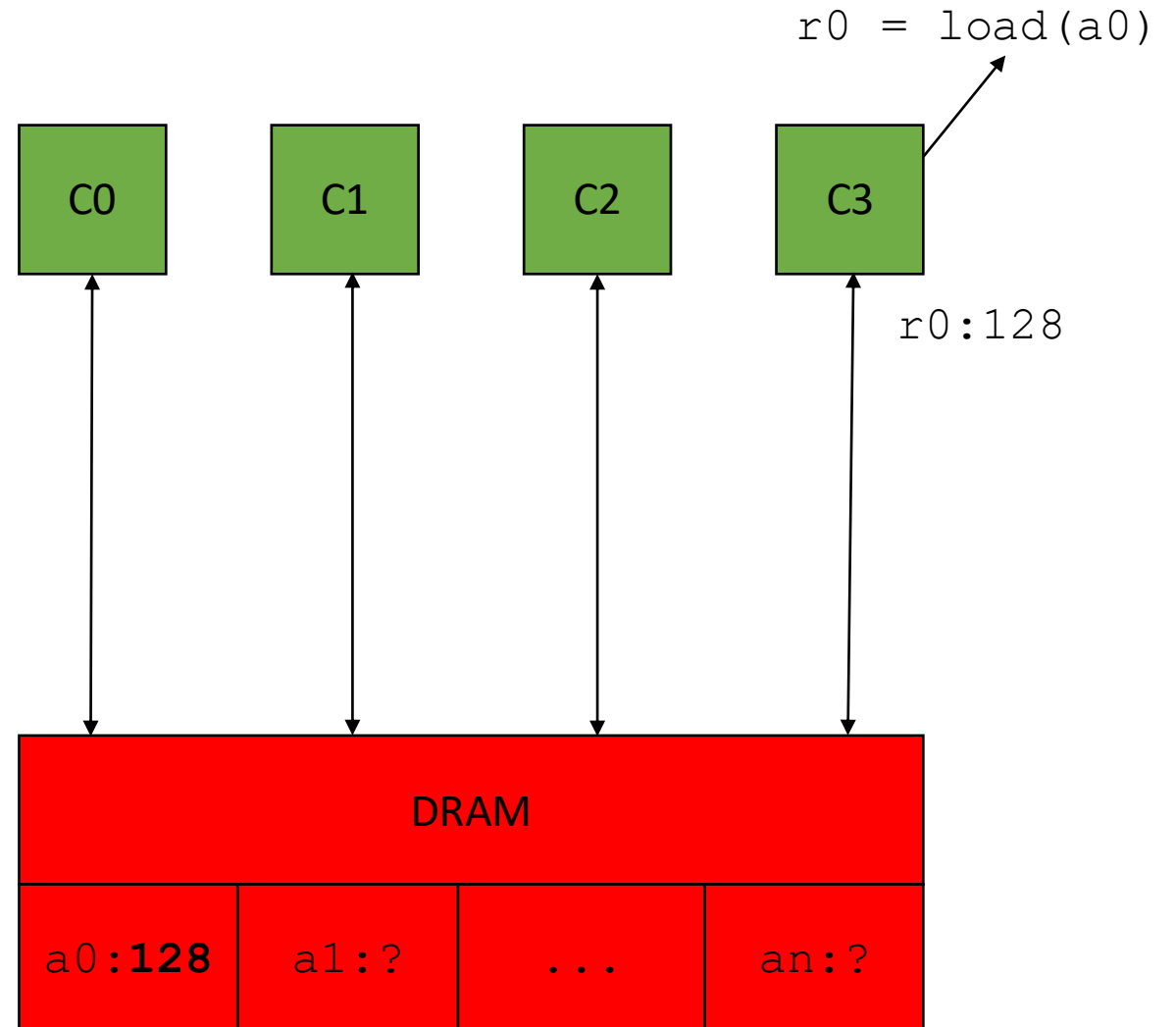
Main memory

Problem solved!

Threads can communicate!

reading a value takes ~200 cycles

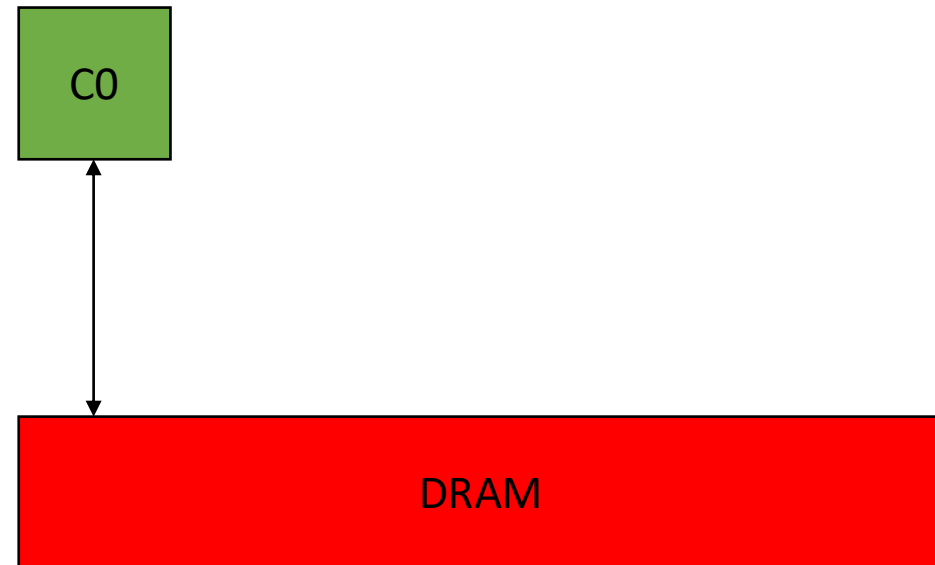
Bad for parallelism, but
also really bad for sequential
code (which we optimized for
decades!)



Main memory

```
int increment(int *a) {  
    a[0]++;  
}
```

```
%5 = load i32, i32* %4  
%6 = add nsw i32 %5, 1  
store i32 %6, i32* %4
```

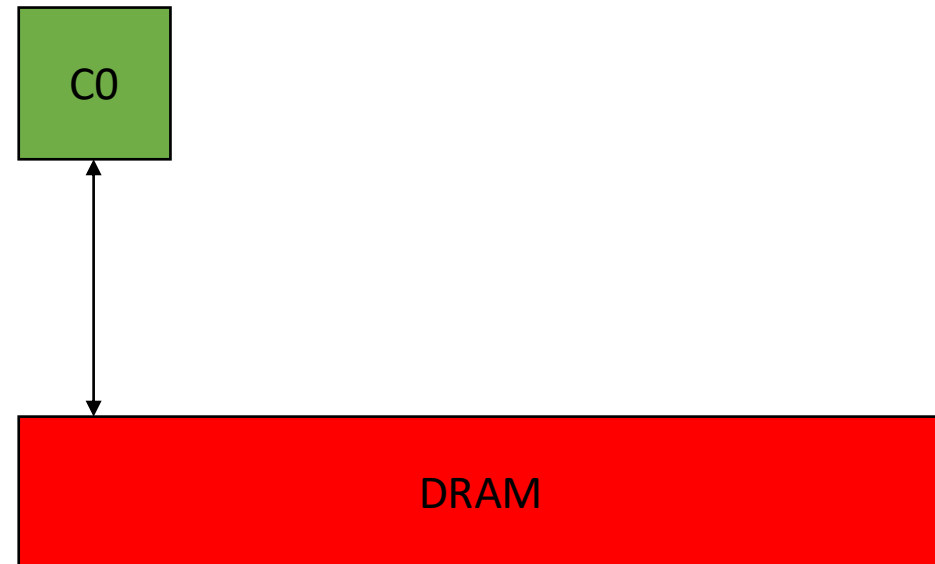


Main memory

```
int increment(int *a) {  
    a[0]++;  
}
```

```
%5 = load i32, i32* %4  
%6 = add nsw i32 %5, 1  
store i32 %6, i32* %4
```

200 cycles



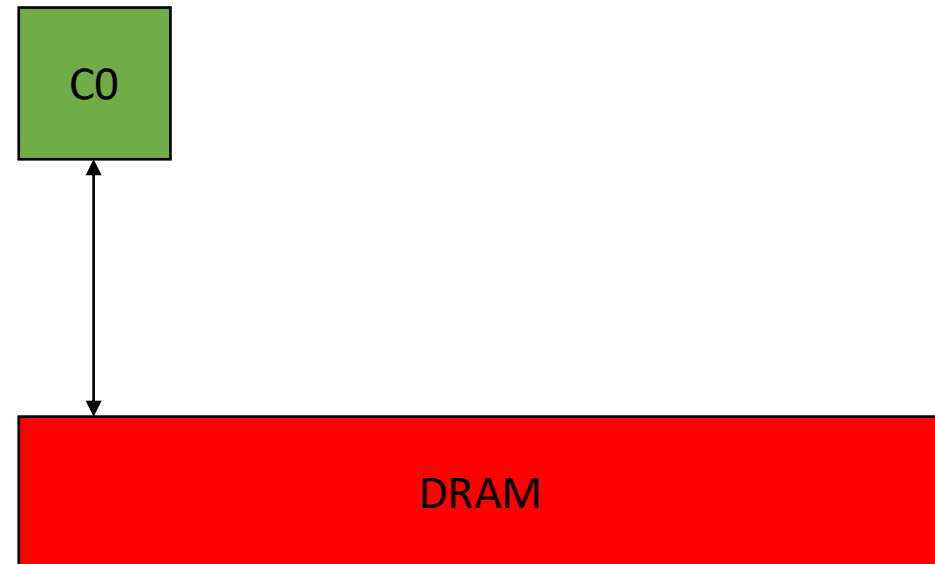
Main memory

```
int increment(int *a) {  
    a[0]++;  
}
```

```
%5 = load i32, i32* %4  
%6 = add nsw i32 %5, 1  
store i32 %6, i32* %4
```

200 cycles

1 cycles



Main memory

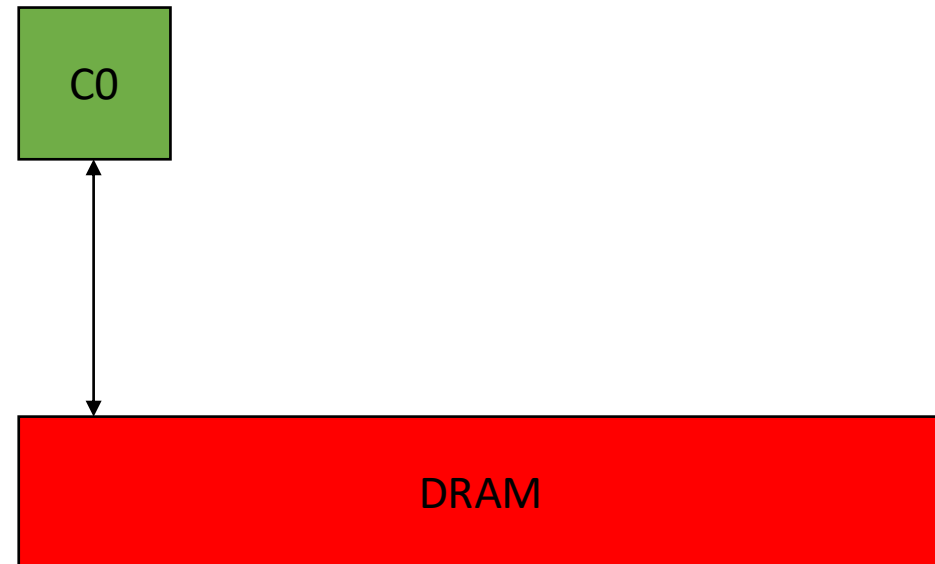
```
int increment(int *a) {  
    a[0]++;  
}
```

```
%5 = load i32, i32* %4  
%6 = add nsw i32 %5, 1  
store i32 %6, i32* %4
```

200 cycles

1 cycles

200 cycles



Main memory

```
int increment(int *a) {  
    a[0]++;  
}
```

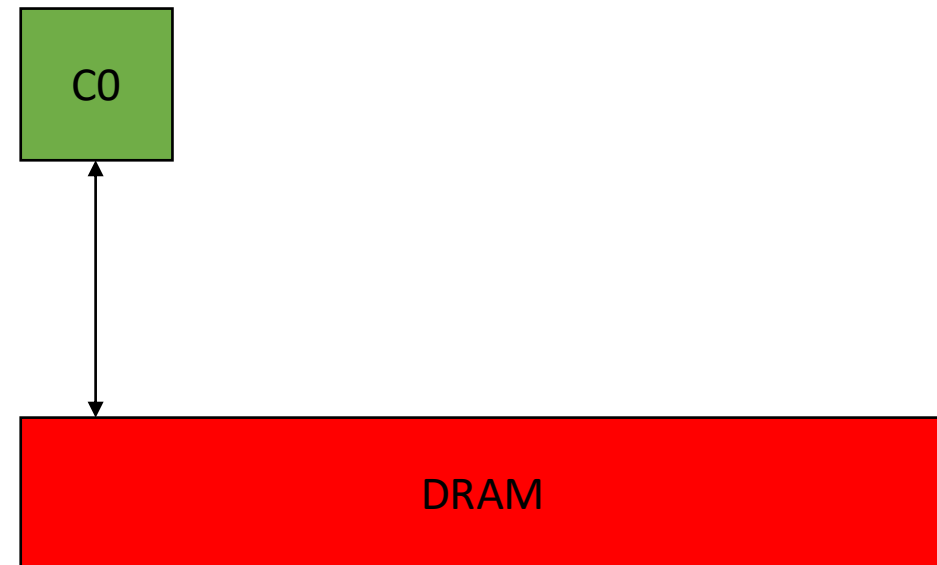
```
%5 = load i32, i32* %4  
%6 = add nsw i32 %5, 1  
store i32 %6, i32* %4
```

200 cycles

1 cycles

200 cycles

401 cycles



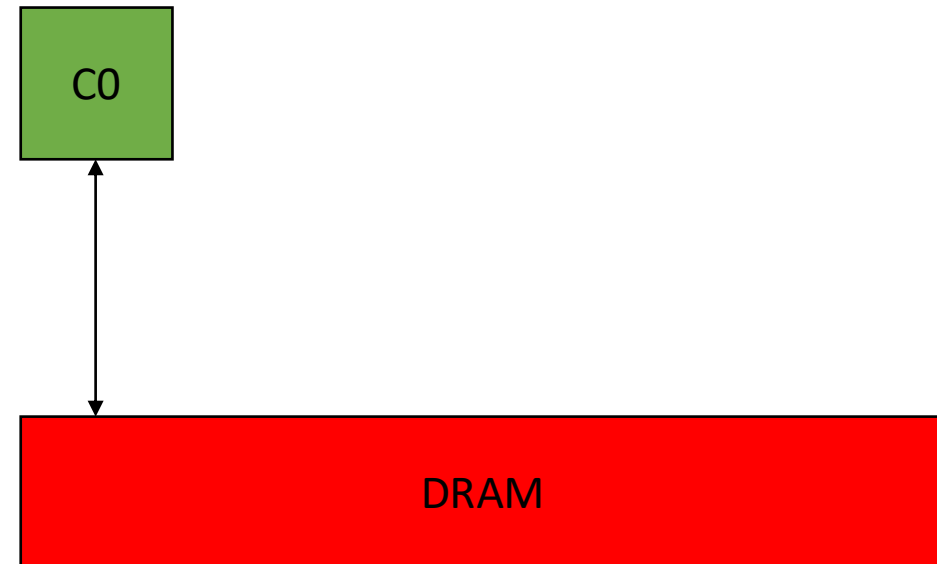
Main memory

```
int increment(int *a) {  
    a[0]++;  
}
```

```
int x = 0;  
for (int i = 0; i < 100; i++) {  
    increment(&x);  
}
```

```
%5 = load i32, i32* %4    200 cycles  
%6 = add nsw i32 %5, 1    1 cycles  
store i32 %6, i32* %4    200 cycles
```

401 cycles



Main memory

```
int increment(int *a) {  
    a[0]++;  
}
```

```
int x = 0;  
for (int i = 0; i < 100; i++) {  
    increment(&x);  
}
```

40100 cycles!

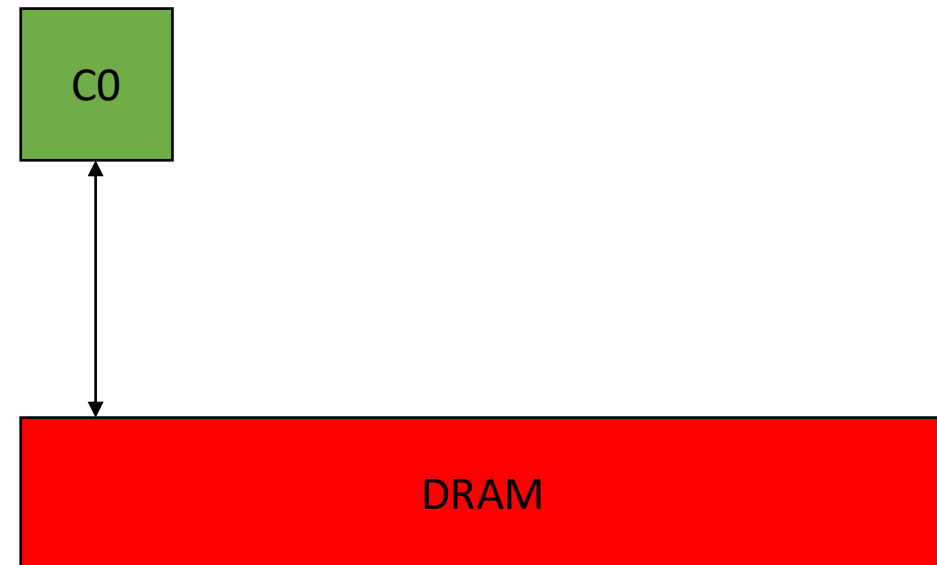
```
%5 = load i32, i32* %4  
%6 = add nsw i32 %5, 1  
store i32 %6, i32* %4
```

200 cycles

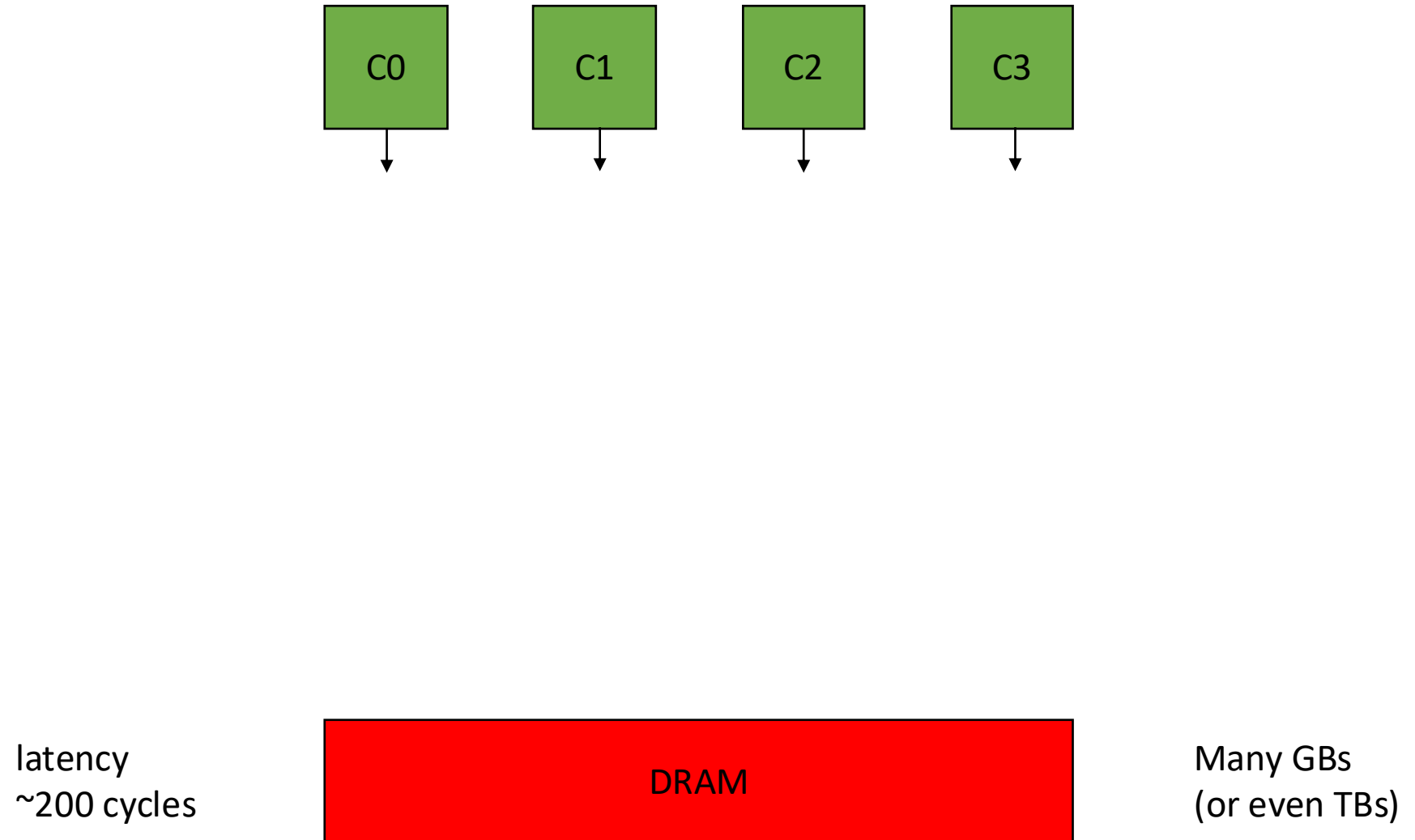
1 cycles

200 cycles

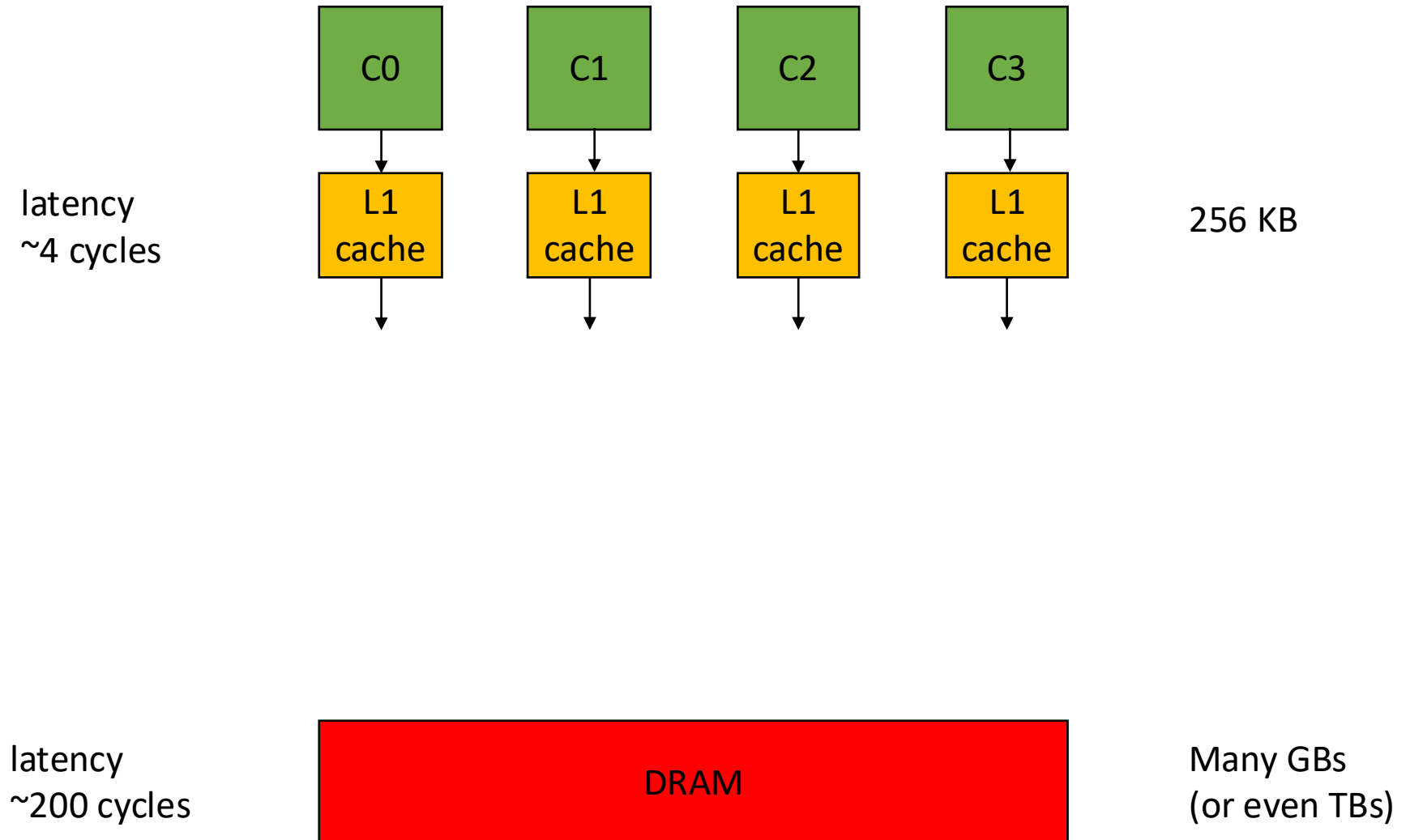
401 cycles



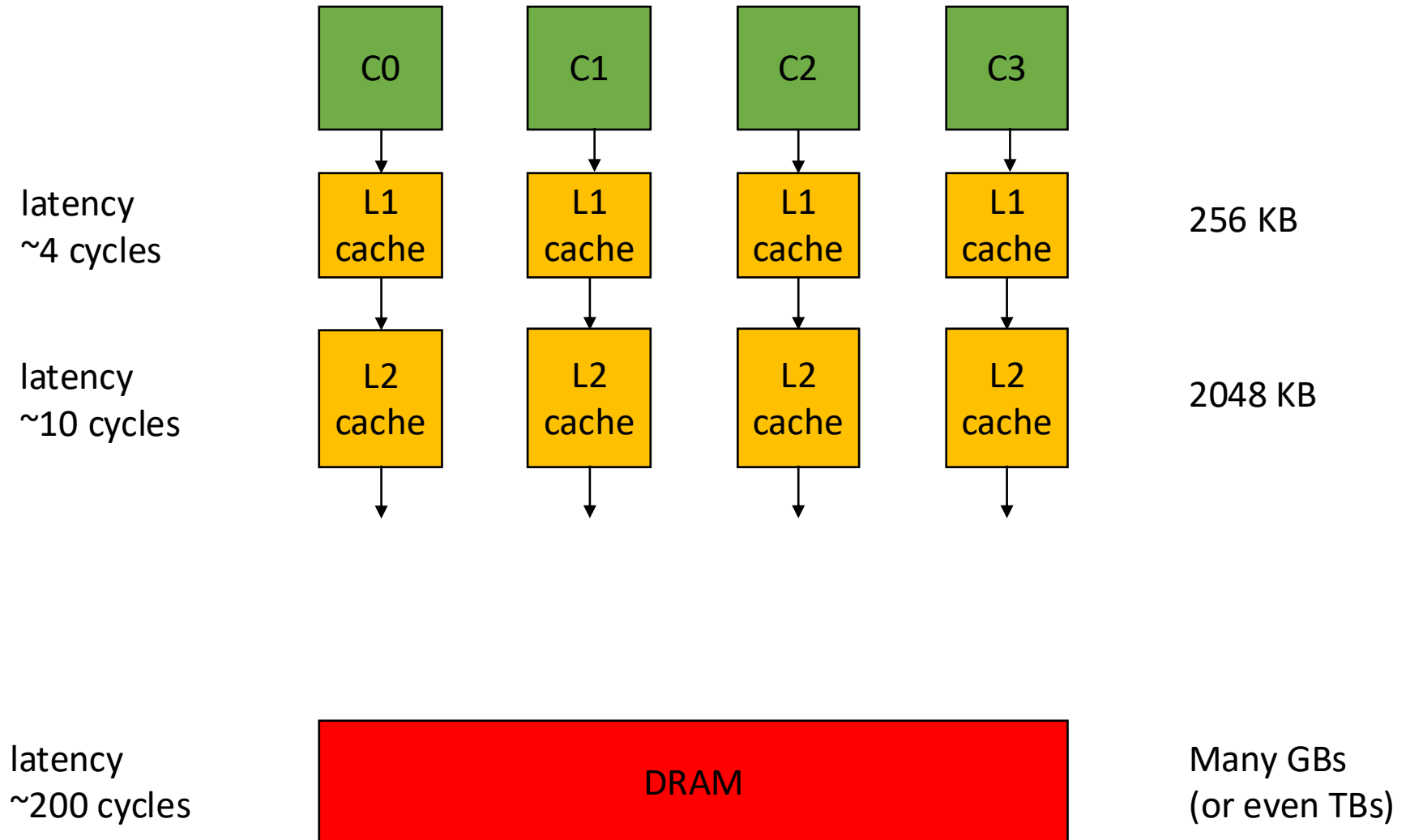
Caches



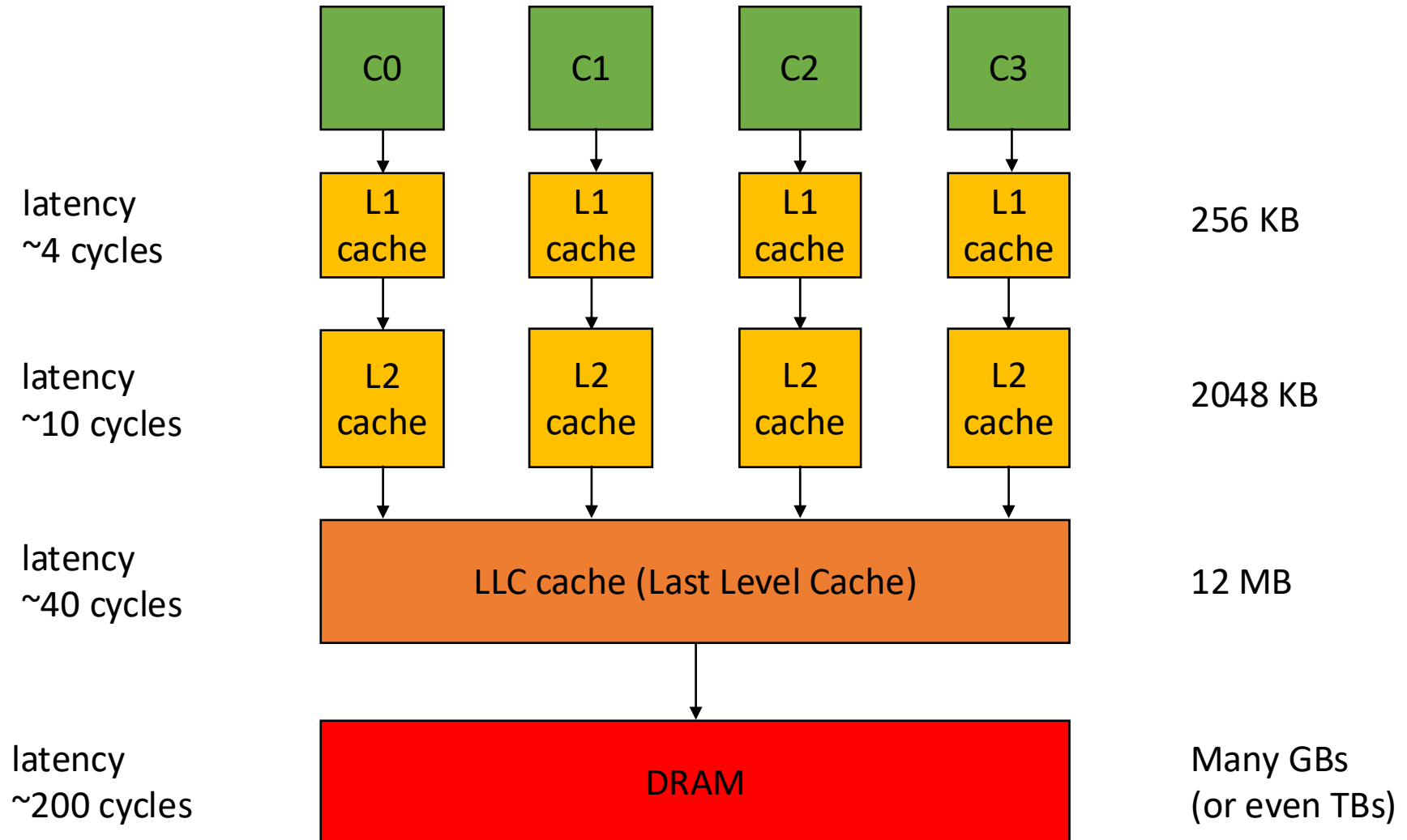
Caches



Caches



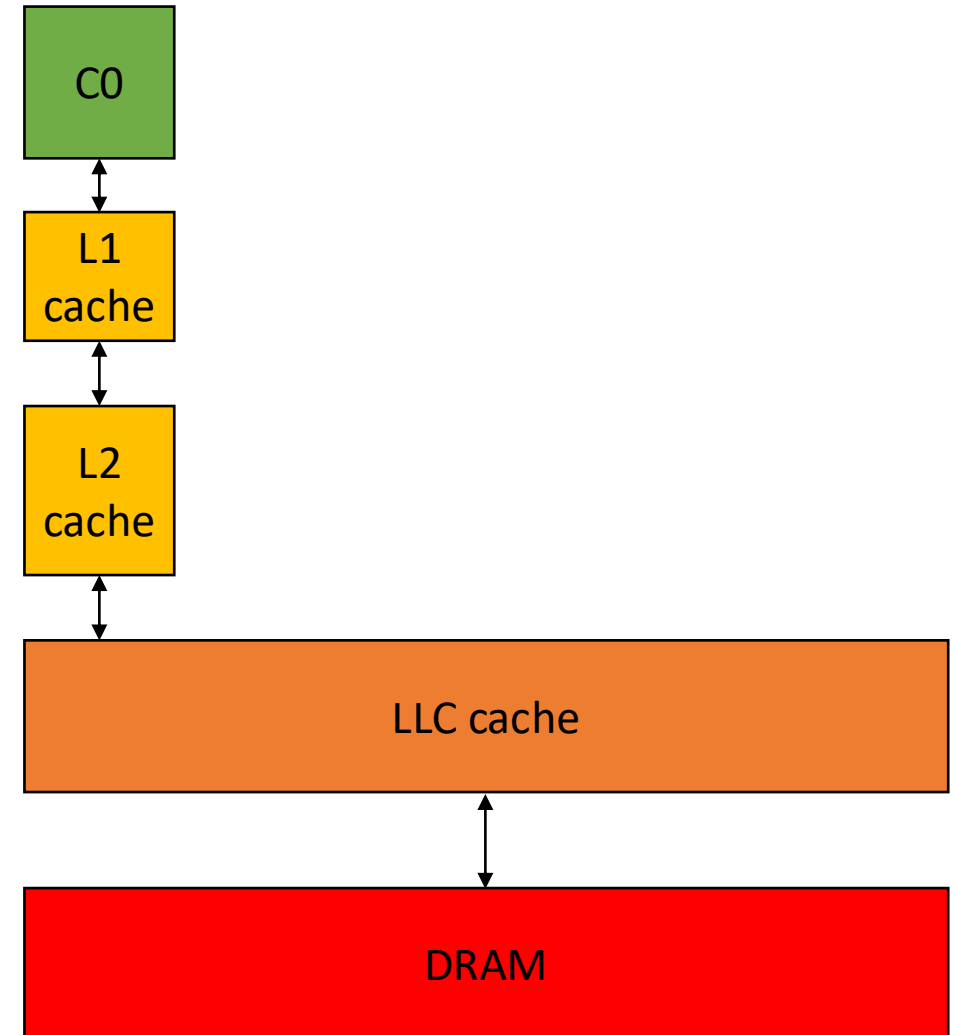
Caches



Caches

```
int increment(int *a) {  
    a[0]++;  
}
```

```
%5 = load i32, i32* %4  
%6 = add nsw i32 %5, 1  
store i32 %6, i32* %4
```



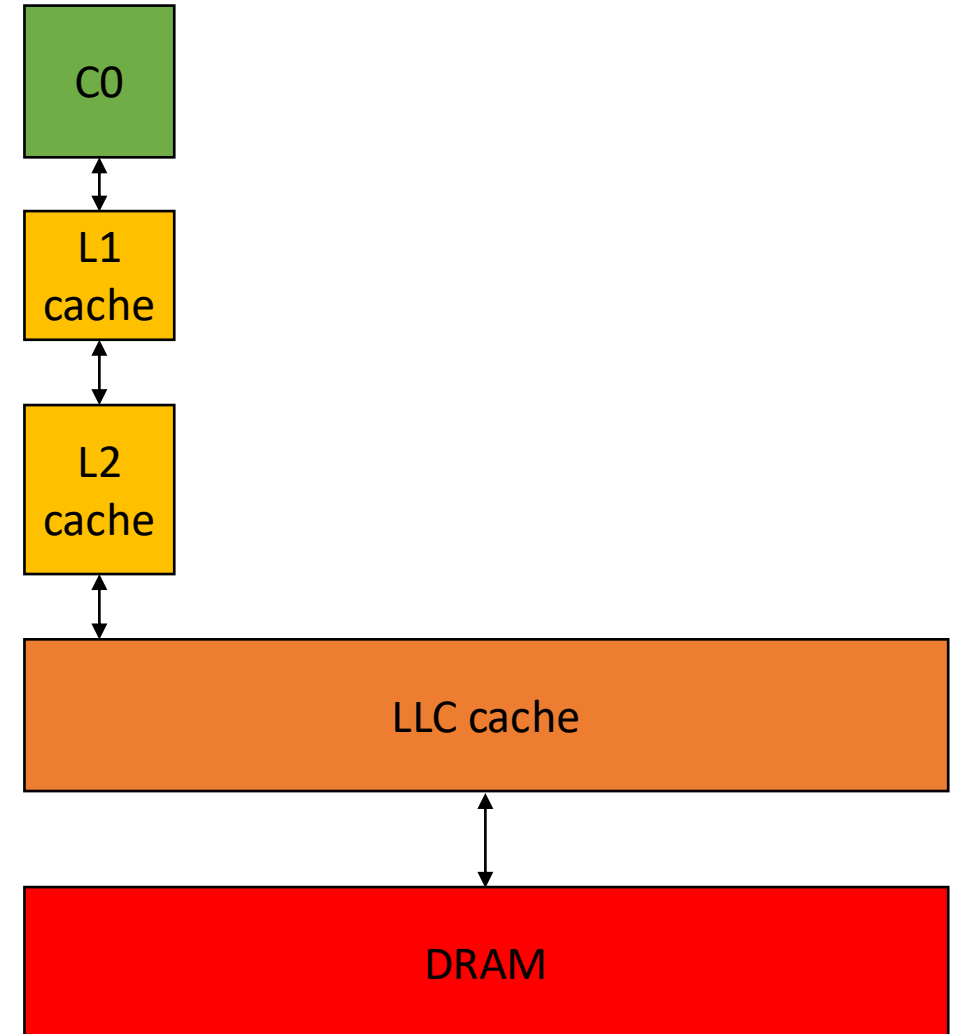
Caches

```
int increment(int *a) {  
    a[0]++;  
}
```

```
%5 = load i32, i32* %4  
%6 = add nsw i32 %5, 1  
store i32 %6, i32* %4
```

4 cycles

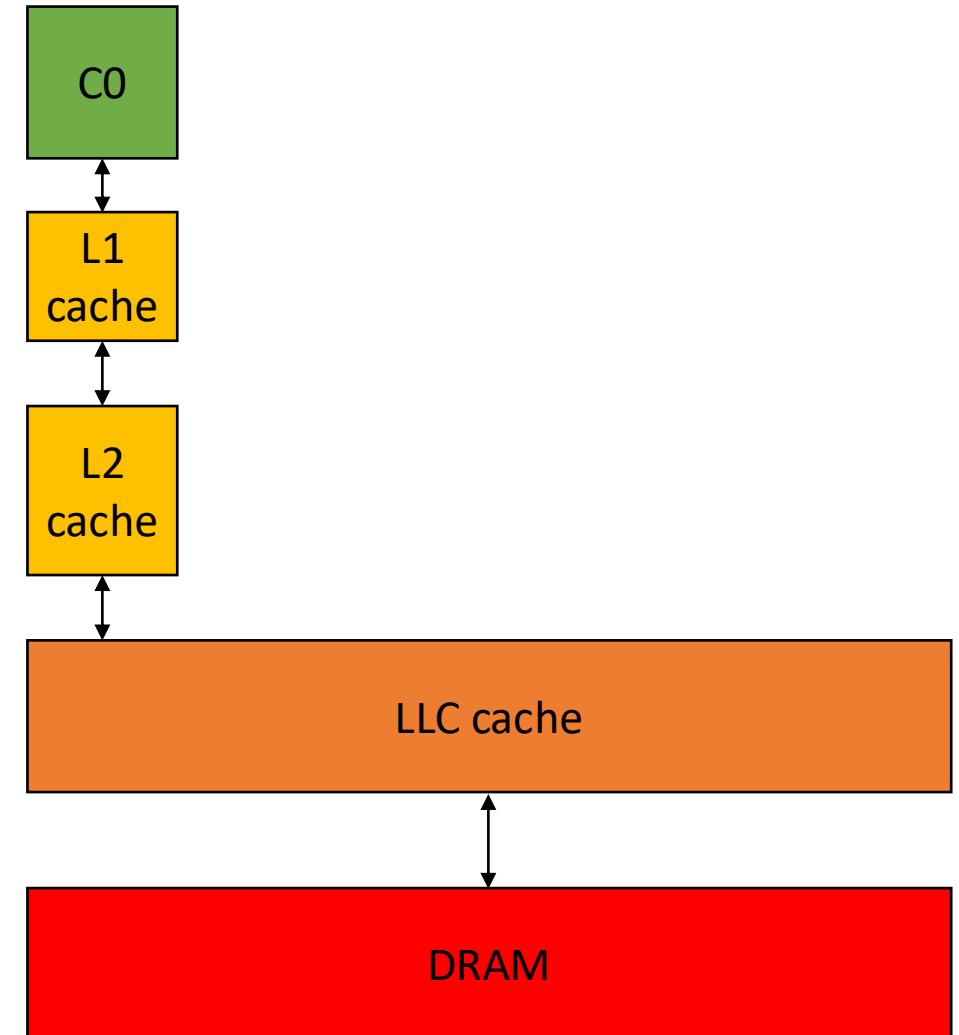
Assuming the value is in the cache!



Caches

```
int increment(int *a) {  
    a[0]++;  
}
```

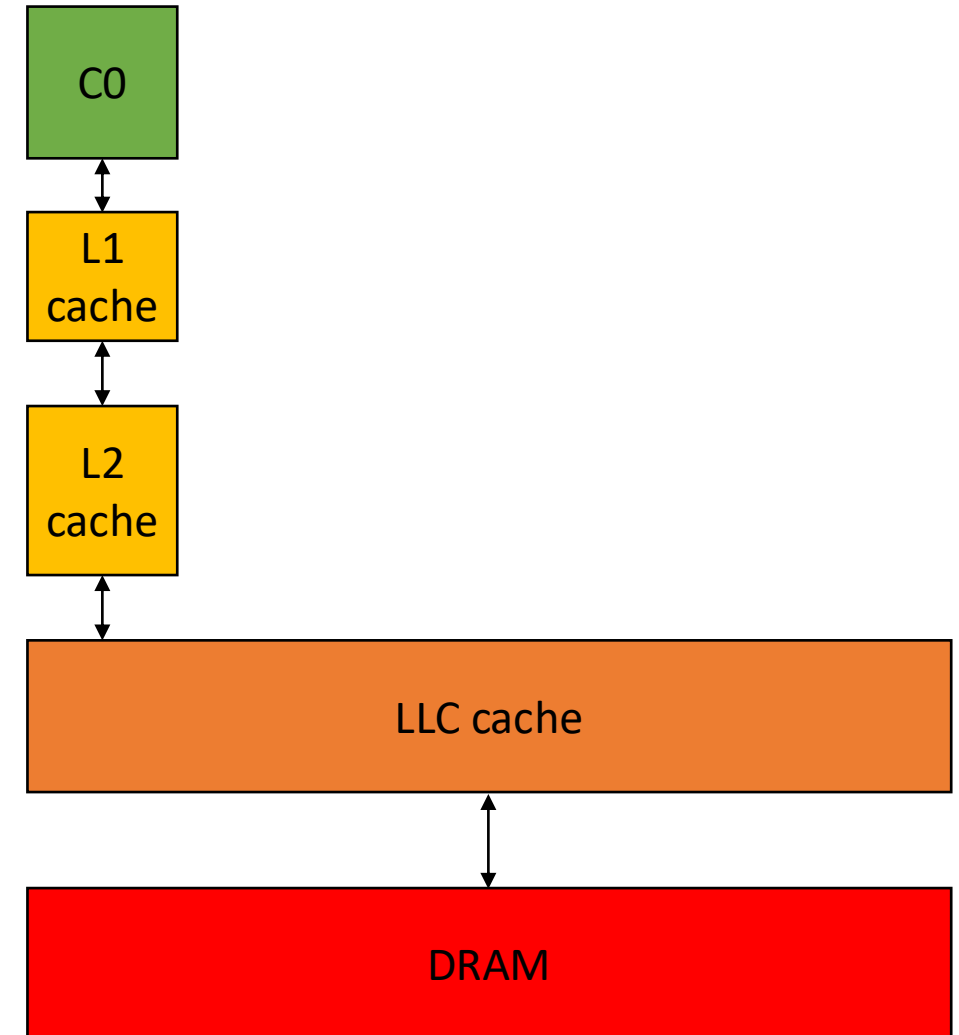
%5 = load i32, i32* %4 4 cycles
%6 = add nsw i32 %5, 1 1 cycles
store i32 %6, i32* %4



Caches

```
int increment(int *a) {  
    a[0]++;  
}
```

%5 = load i32, i32* %4	4 cycles
%6 = add nsw i32 %5, 1	1 cycles
store i32 %6, i32* %4	4 cycles

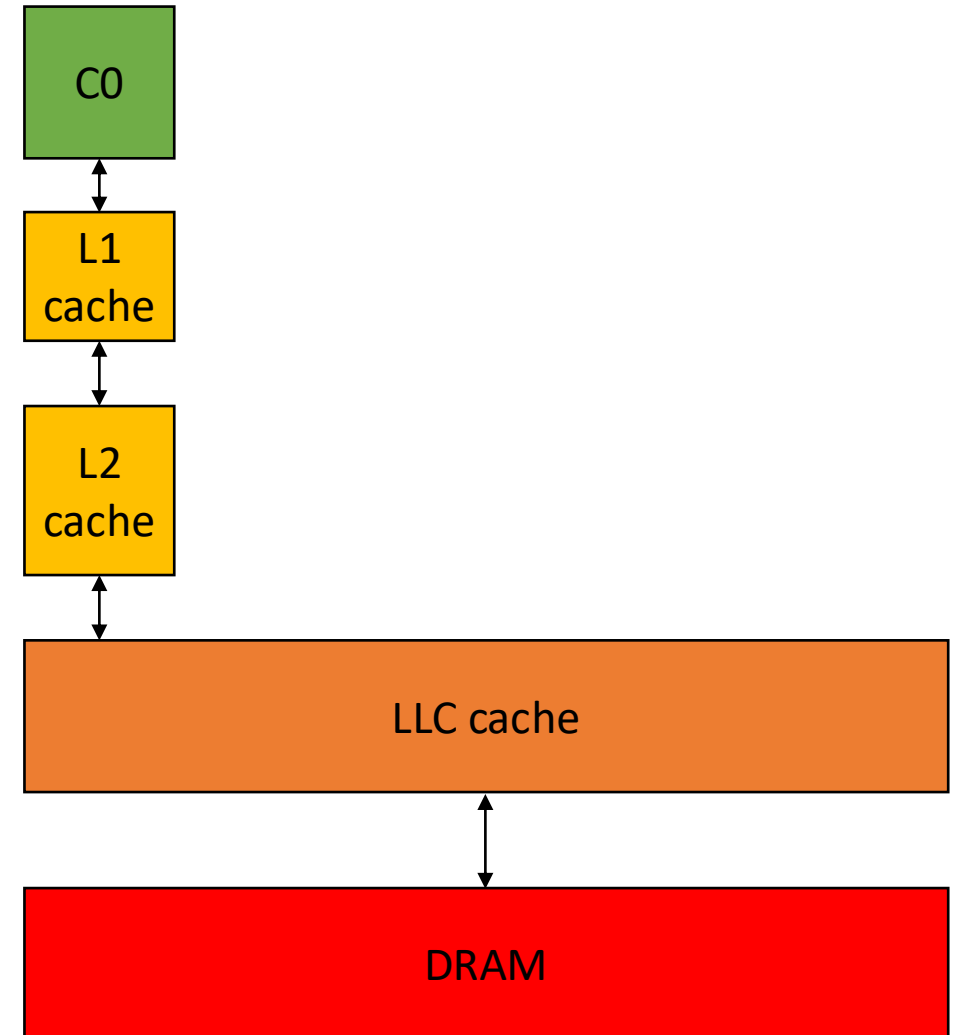


Caches

```
int increment(int *a) {  
    a[0]++;  
}
```

%5 = load i32, i32* %4	4 cycles
%6 = add nsw i32 %5, 1	1 cycles
store i32 %6, i32* %4	4 cycles

9 cycles!



Quick overview of C/++ pointers/memory

Passing arrays in C++

```
int increment(int *a) {  
    a[0]++;  
}
```

```
int increment_alt1(int a[1]) {  
    a[0]++;  
}
```

```
int increment_alt2(int a[]) {  
    a[0]++;  
}
```

Not checked at compile time! but hints can help with compiler optimizations. Also good self documenting code.

Passing pointers

```
int foo0(int *a) {  
    increment(a)  
}
```

pass pointer directly through

```
int foo1(int *a) {  
    increment(&(a[8]))  
}
```

pass an offset of 8

```
int foo2(int *a) {  
    increment(a + 8)  
}
```

another way to pass an offset of 8

Memory Allocation

```
int allocate_int_array0() {  
    int ar[16];  
}
```

Stack allocation

```
int allocate_int_array1() {  
    int *ar = new int[16];  
    delete[] ar;  
}
```

Heap allocation, C++ style

```
int allocate_int_array2() {  
    int *ar = (int*)malloc(sizeof(int)*16);  
    free(ar);  
}
```

Heap allocation, C style

Cache lines

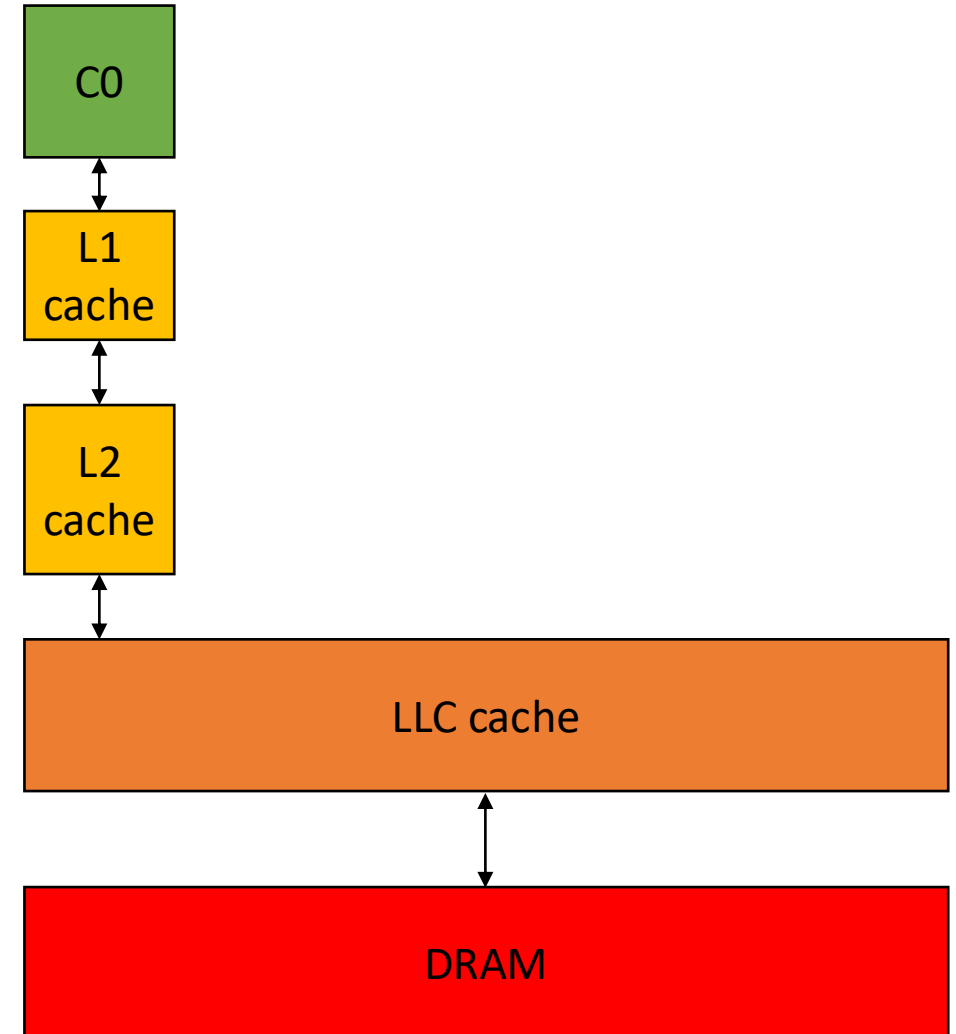
- Cache line size for x86: 64 bytes:
 - 64 chars
 - 32 shorts
 - 16 float or int
 - 8 double or long

Caches

```
int increment(int *a) {  
    a[0]++;  
}
```

```
%5 = load i32, i32* %4  
%6 = add nsw i32 %5, 1  
store i32 %6, i32* %4
```

Assume a[0] is not in the cache



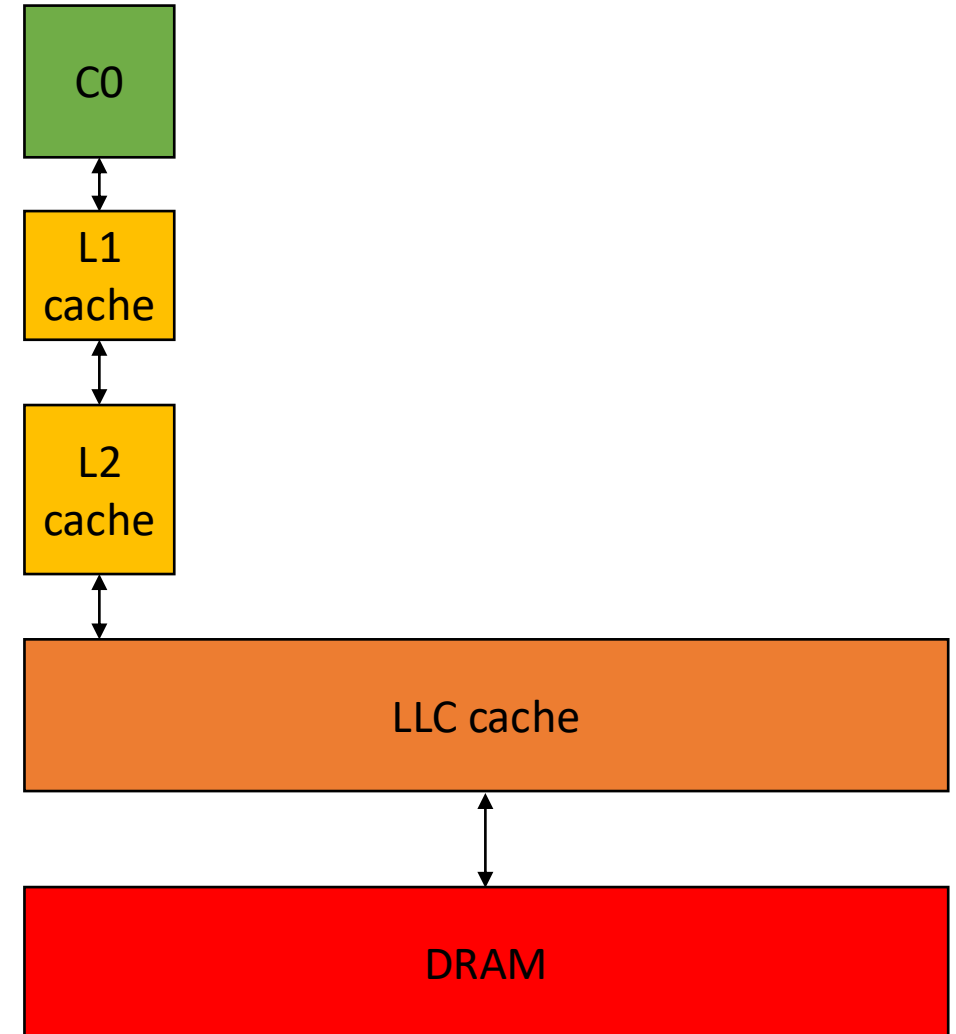
Caches

```
int increment(int *a) {  
    a[0]++;  
}
```

```
%5 = load i32, i32* %4  
%6 = add nsw i32 %5, 1  
store i32 %6, i32* %4
```

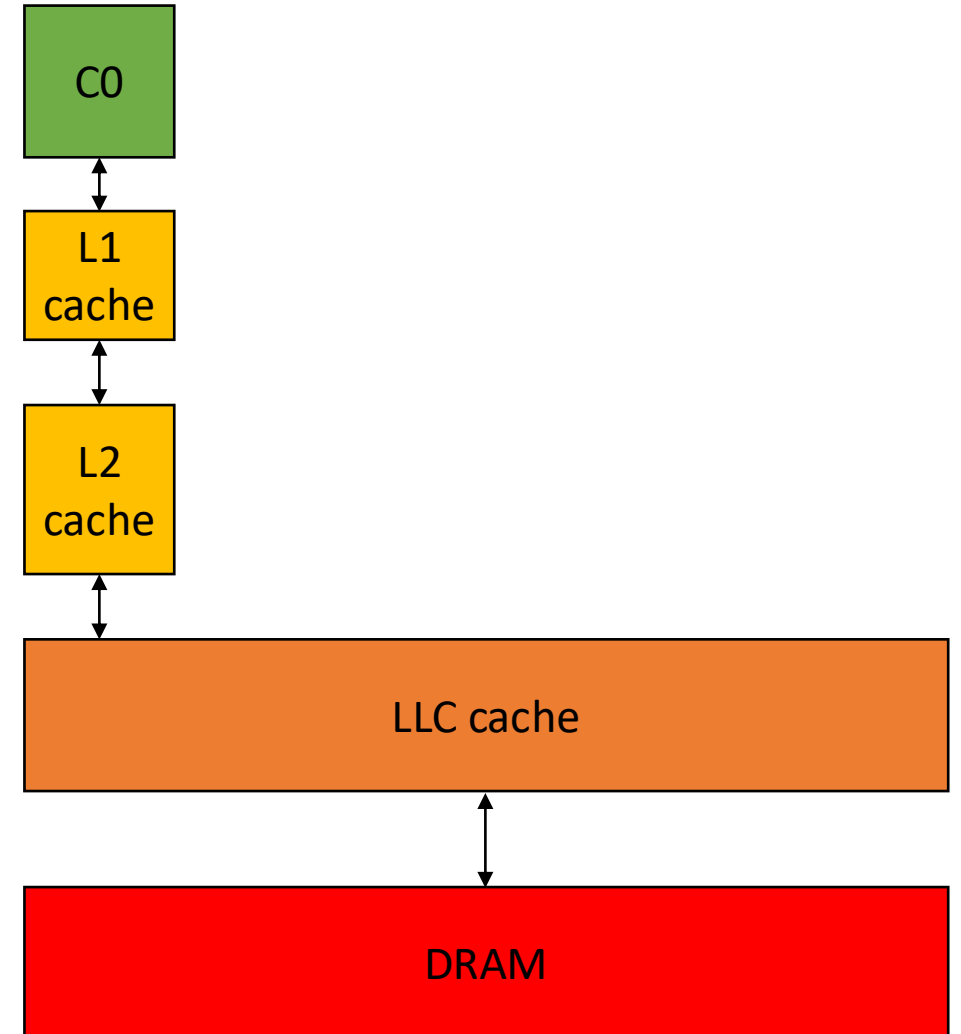
$a[0] - a[15]$

Assume $a[0]$ is not in the cache



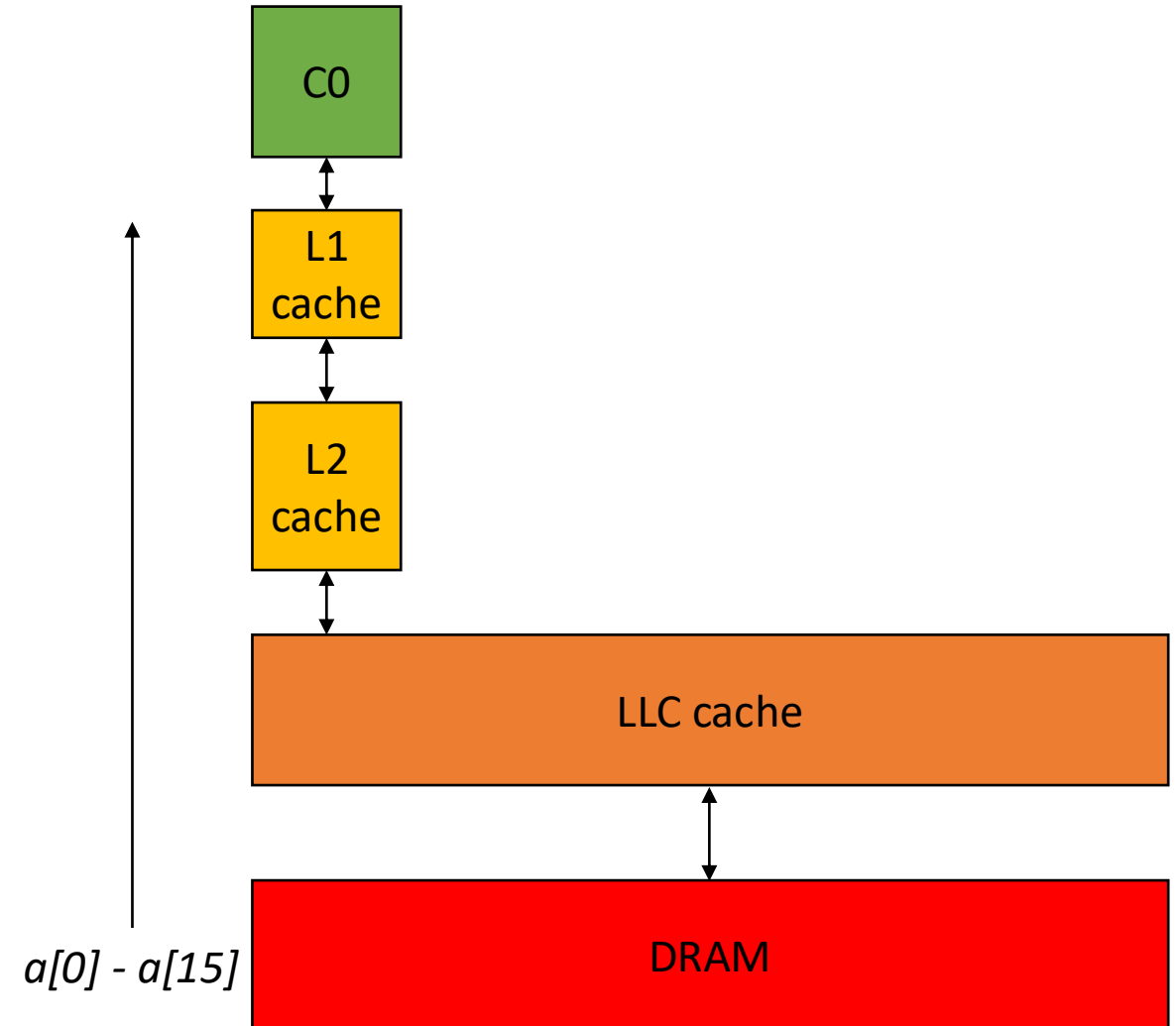
Caches

```
int increment_several(int *a) {  
    a[0]++;  
    a[15]++;  
    a[16]++;  
}
```



Caches

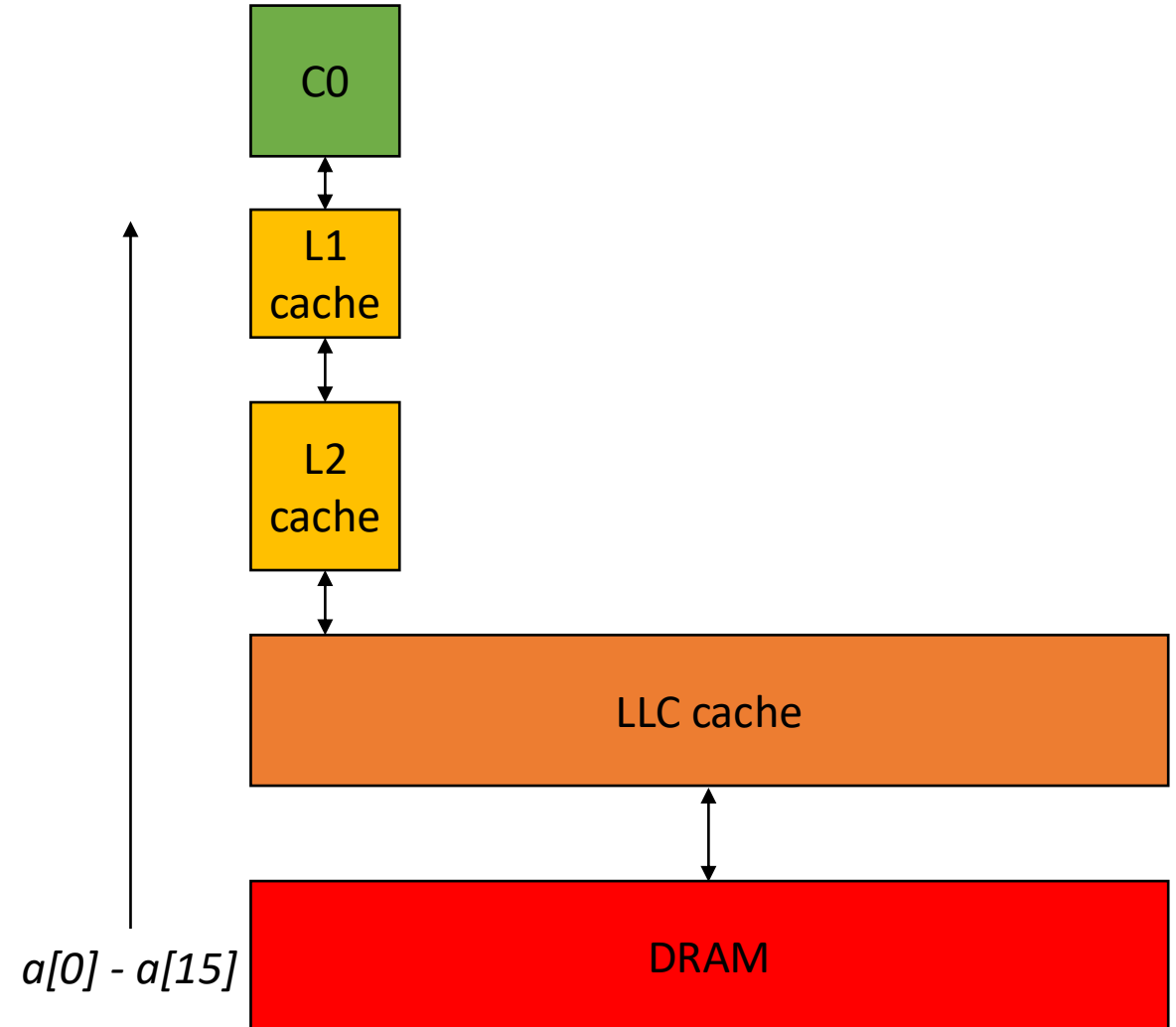
```
int increment_several(int *a) {  
    a[0]++;  
    a[15]++;  
    a[16]++;  
}
```



Caches

```
int increment_several(int *a) {  
    a[0]++;  
    a[15]++;  
    a[16]++;  
}
```

will be a hit because we've loaded a[0] cache line

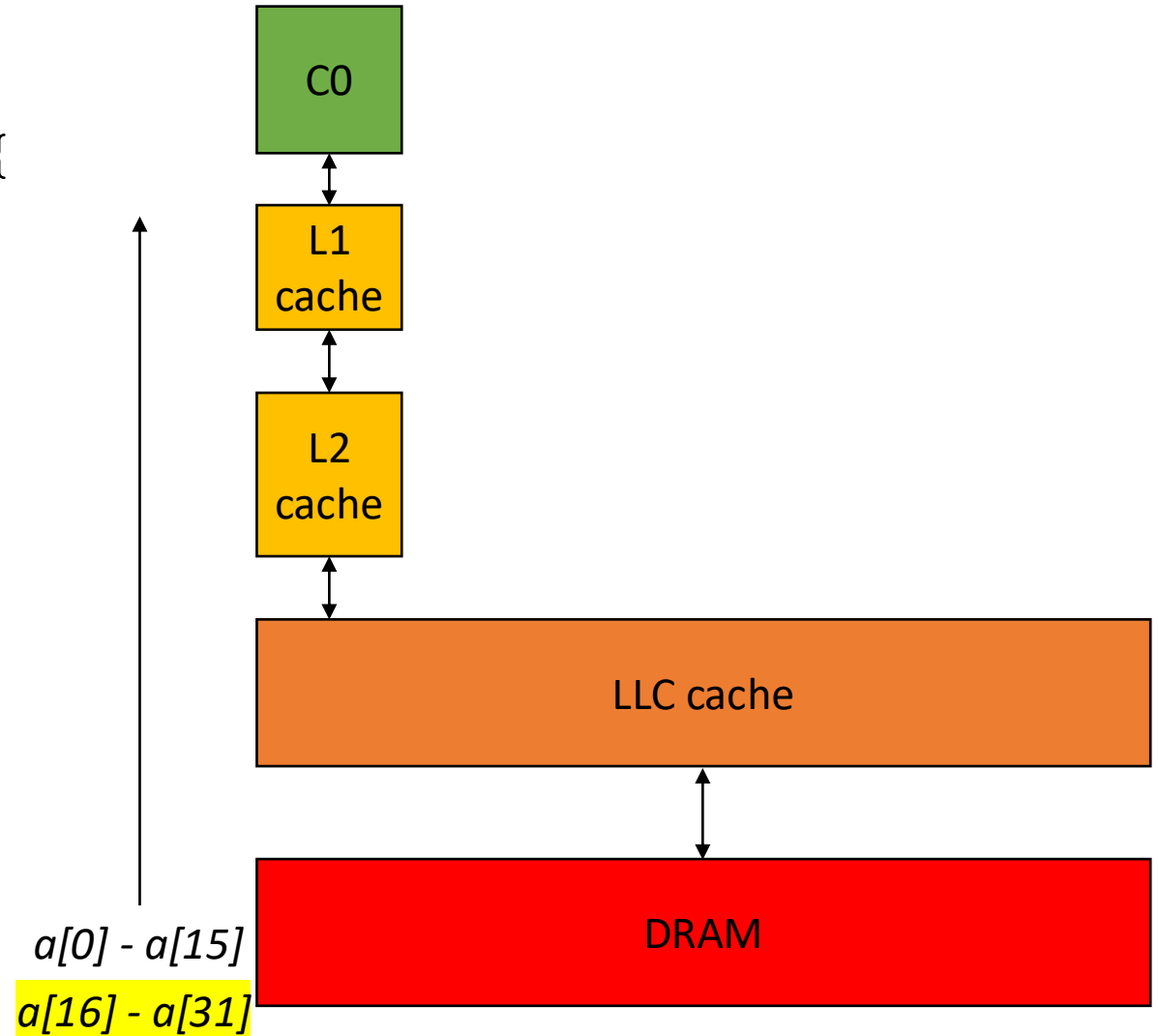


Caches

```
int increment_several(int *a) {  
    a[0]++;  
    a[15]++;  
    a[16]++;  
}
```

Miss

Assume a[0] is not in the cache

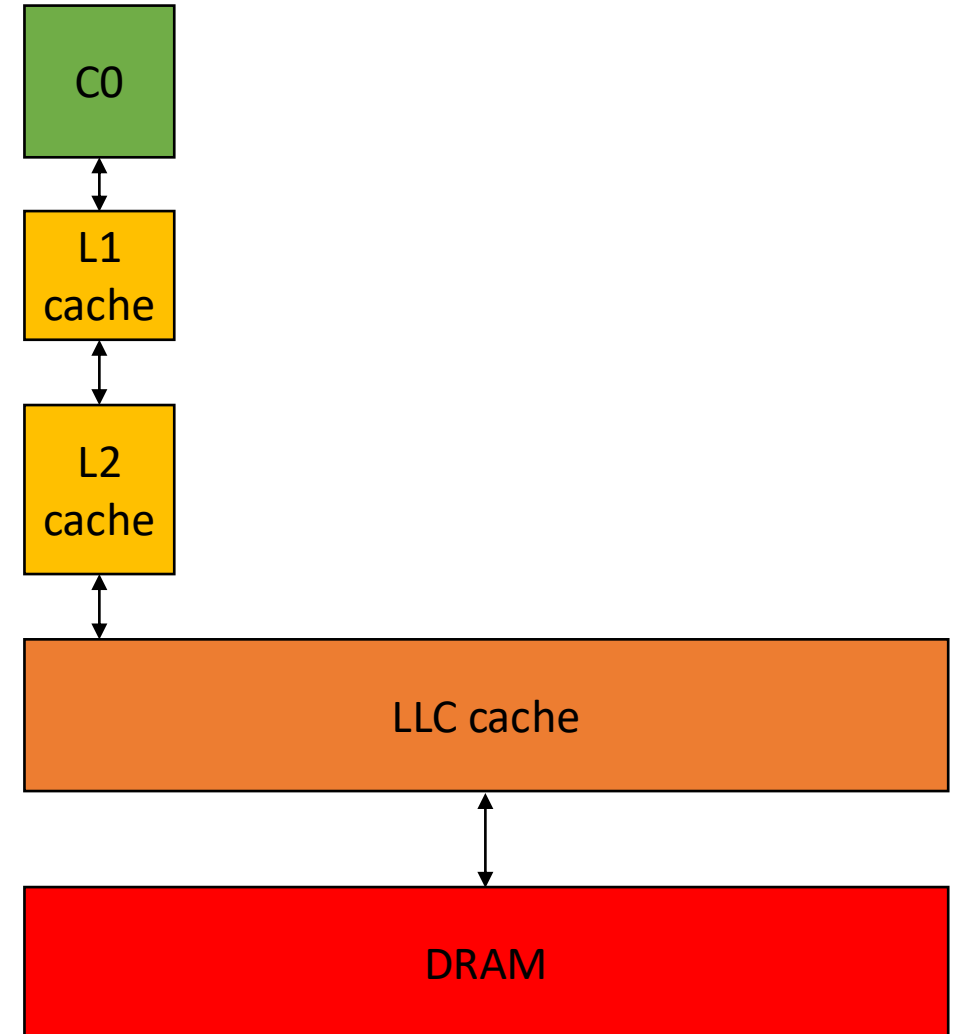


Cache alignment

```
int increment_several(int *b) {  
    b[0]++;  
    b[15]++;  
}
```

```
int foo(int *a) {  
    increment_several(&(a[8]))  
}
```

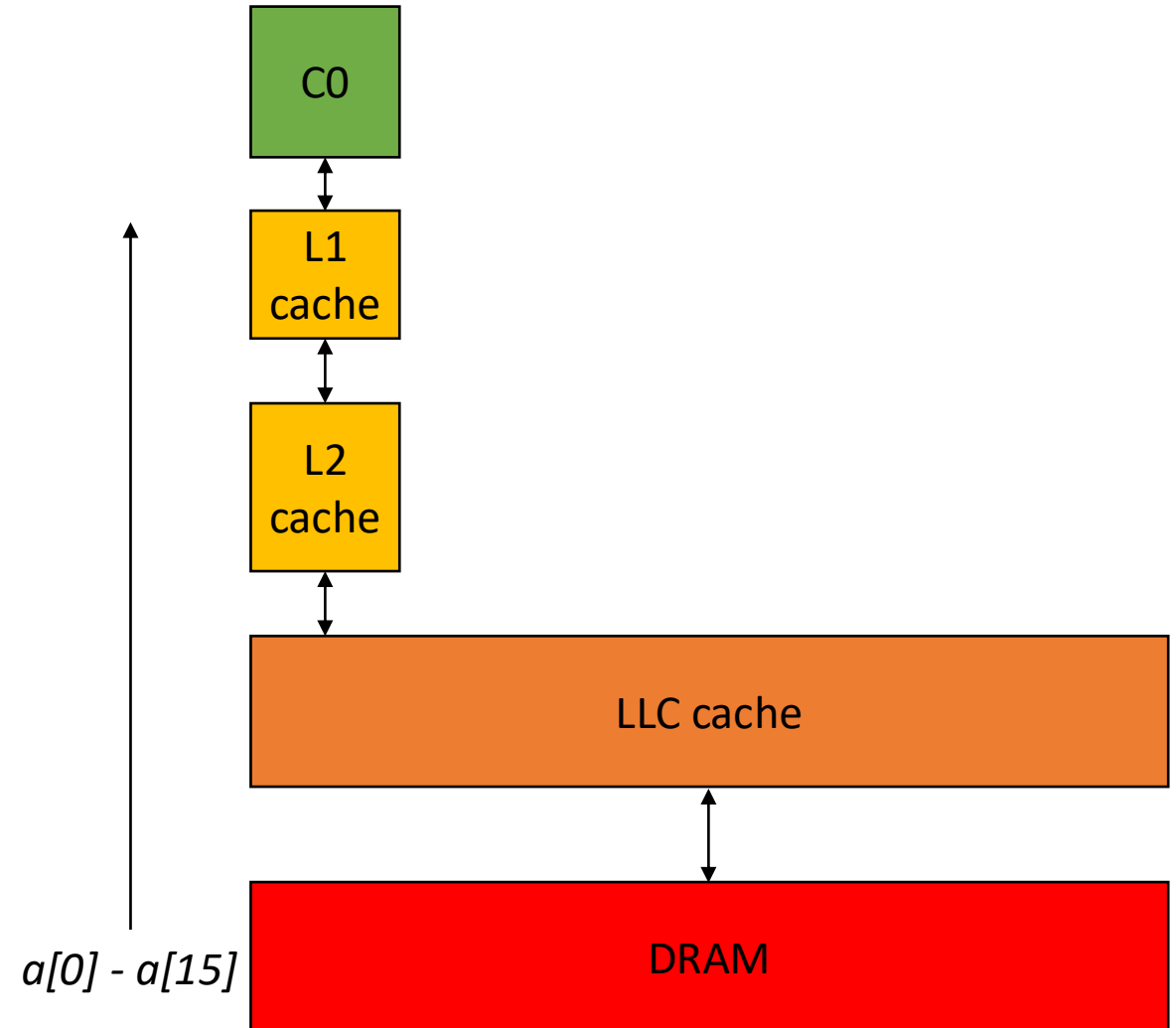
Assume a[0] is not in the cache



Cache alignment

```
int increment_several(int *b) {  
    b[0]++;  
    b[15]++;  
}  
  
int foo(int *a) {  
    increment_several(&(a[8]))  
}
```

Assume $a[0]$ is not in the cache



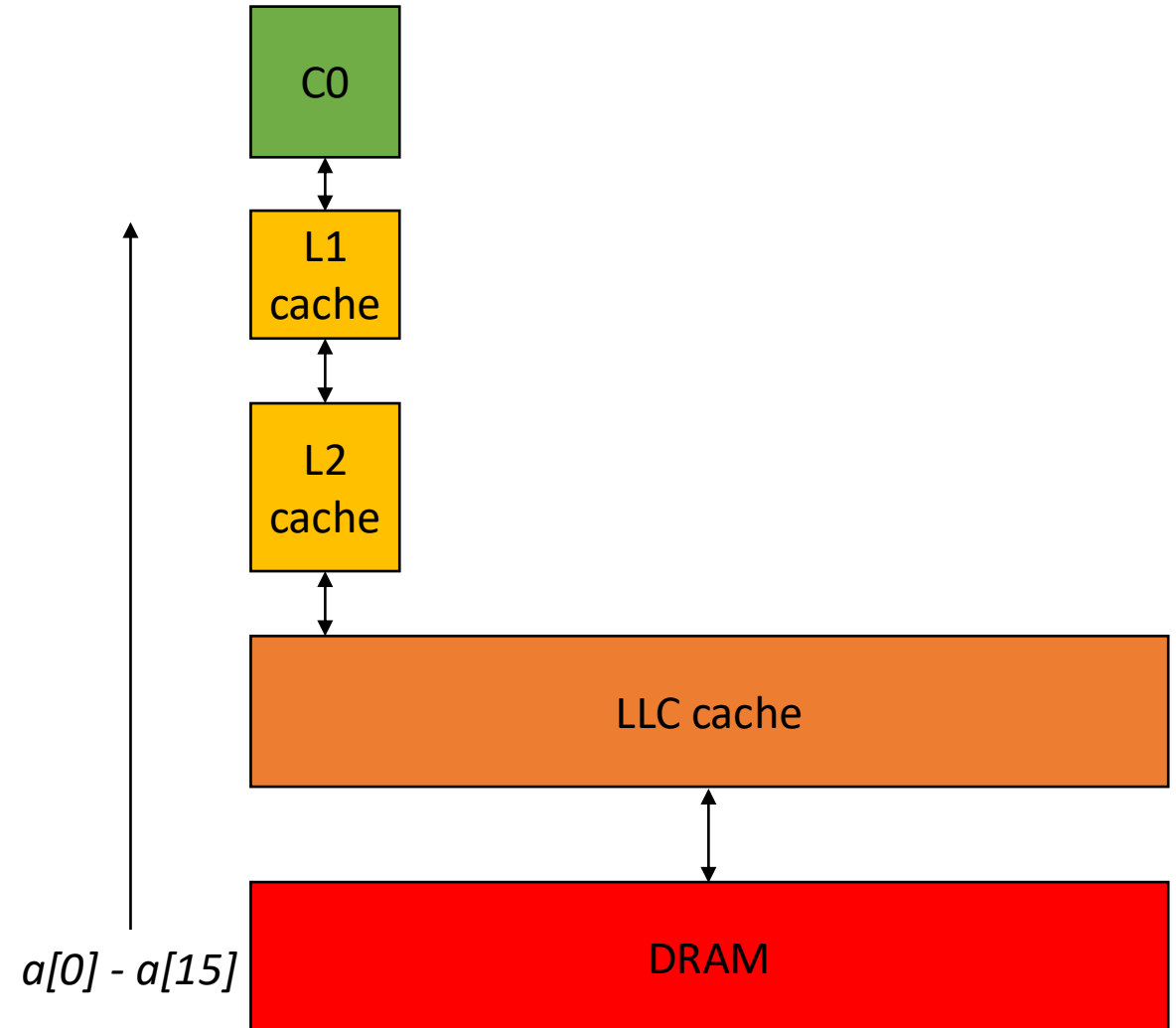
Cache alignment

```
int increment_several(int *b) {  
    b[0]++;  
    b[15]++;  
}
```

```
int foo(int *a) {  
    increment_several(&(a[8]))  
}
```

This loads a[8]

Assume a[0] is not in the cache



Cache alignment

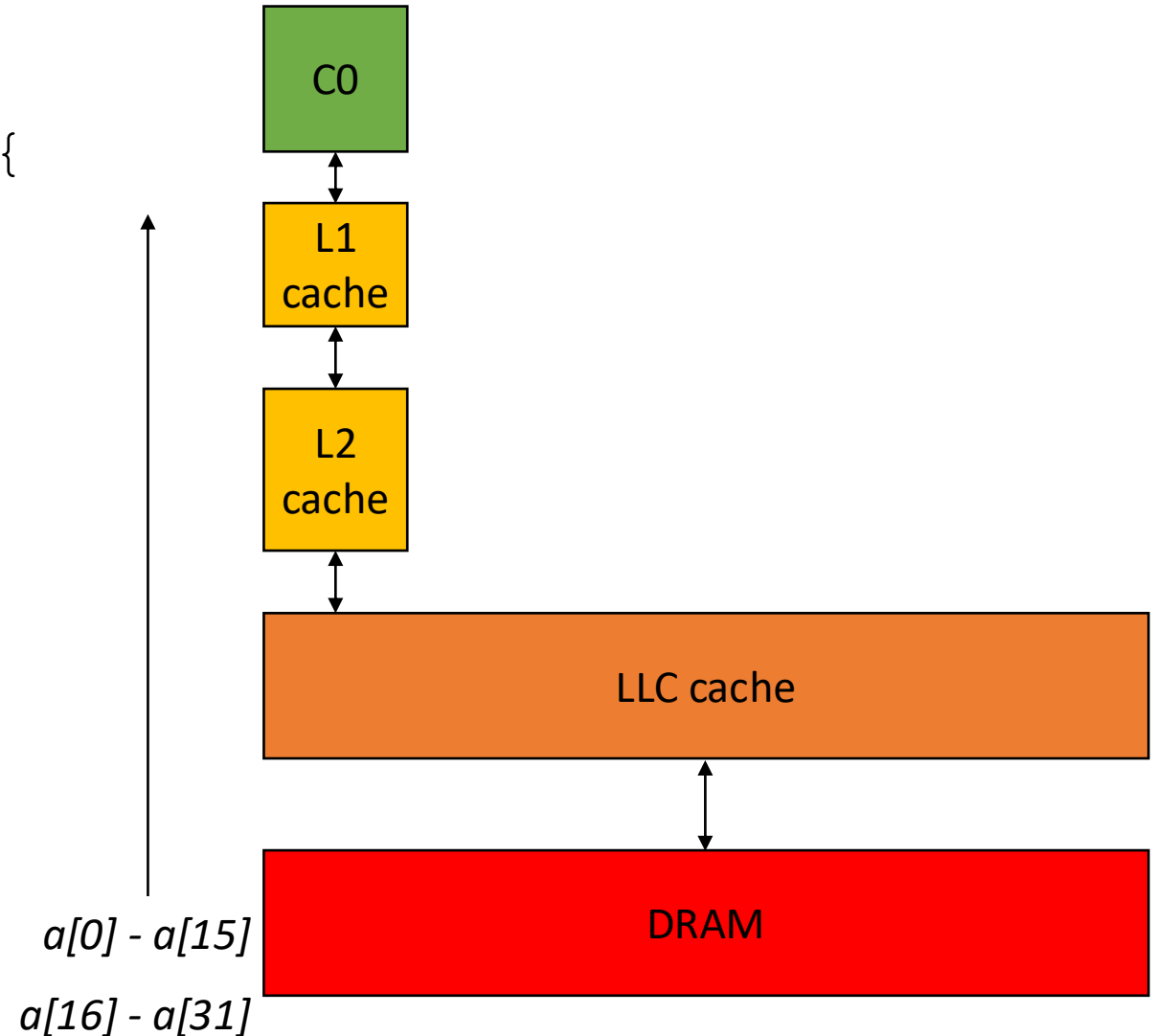
```
int increment_several(int *b) {  
    b[0]++;  
    b[15]++;  
}
```

```
int foo(int *a) {  
    increment_several(&(a[8]))  
}
```

This loads a[8]

This loads a[23], a miss!

Assume a[0] is not in the cache



Cache alignment

- Malloc typically returns a pointer with “good” alignment.
 - System specific, but will be aligned at least to a cache line, more likely a page
- For very low-level programming you can use special aligned malloc functions
- Prefetchers will also help for many applications (e.g. streaming)

Cache alignment

- Malloc typically returns a pointer with “good” alignment.
 - System specific, but will be aligned at least to a cache line, more likely a page
- For very low-level programming you can use special aligned malloc functions
- Prefetchers will also help for many applications (e.g. streaming)

```
for (int i = 0; i < 100; i++) {  
    a[i] += b[i];  
}
```

prefetcher will start collecting consecutive data in the cache if it detects patterns like this.